

算 法 设 计 与 分 析

新型树型索引结构与算法

Chao Zhang

Renmin University of China

Background-B+-Tree回顾



Adaptive Index

- T tree
- ART
- CSB
- CSB+

Learned Index

- RMI

Learned High-Dimensional Index

- Tsunami
- LA-tree

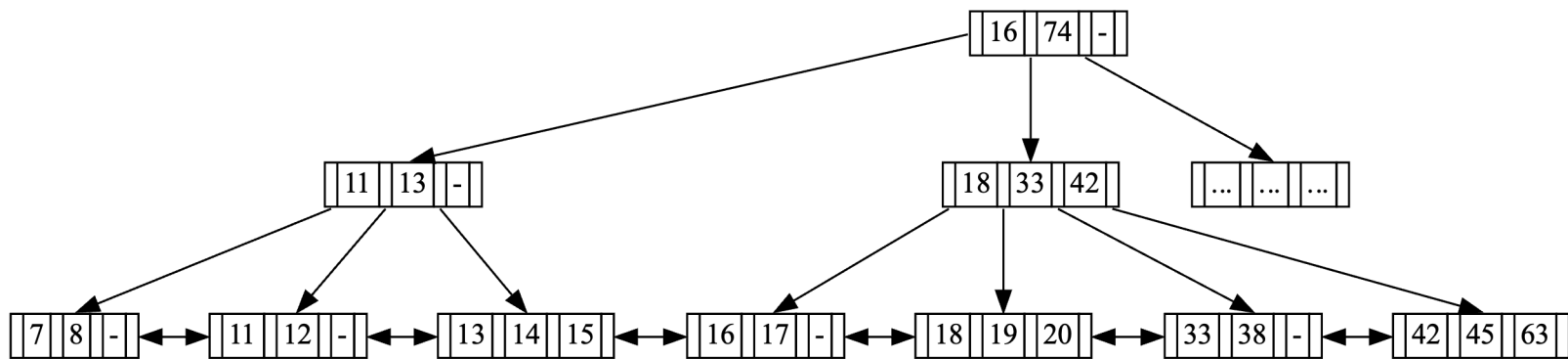
B+树索引结构

➤ 结构：平衡多叉查找树

- 根到所有叶子节点路径长度相同

➤ 扇出 m ：每个节点最多分叉数

- 子节点数目范围 $\left[\left\lceil\frac{m}{2}\right\rceil, m\right]$
- 查找键数目范围 $\left[\left\lceil\frac{m}{2}\right\rceil - 1, m - 1\right]$



B+树索引结构

➤ 节点：页面（磁盘索引）

- 页面缓冲管理器

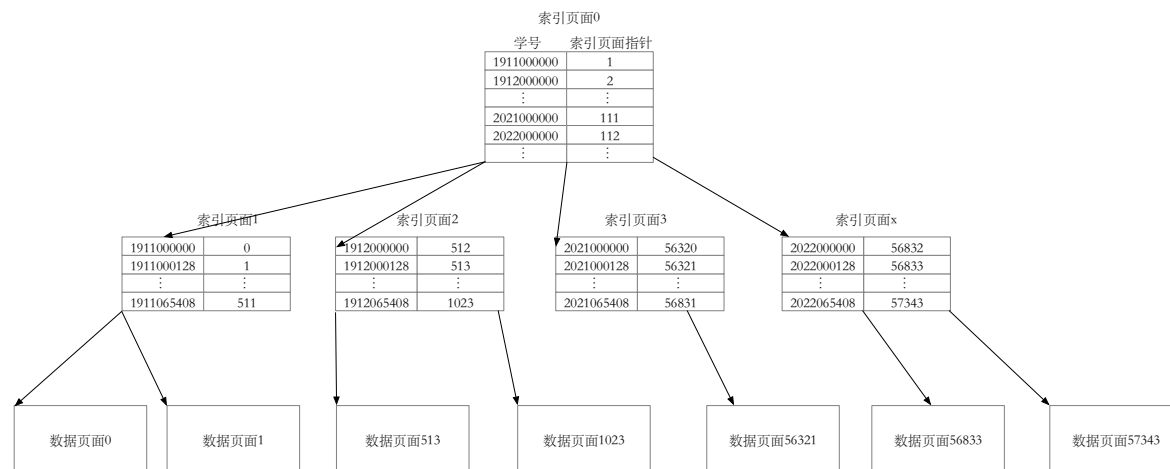
➤ 节点内指针

- 内部节点：索引页面号
- 叶子节点：(数据记录页面号, 数据记录槽号)

➤ $\text{Tree}(P_i) < K_i \leq \text{Tree}(P_{i+1})$

➤ 扇出 m 由页面大小和键值大小决定

- m 通常200 ~ 300
- 10亿条记录仅需要4 ~ 5层
- I/O次数少



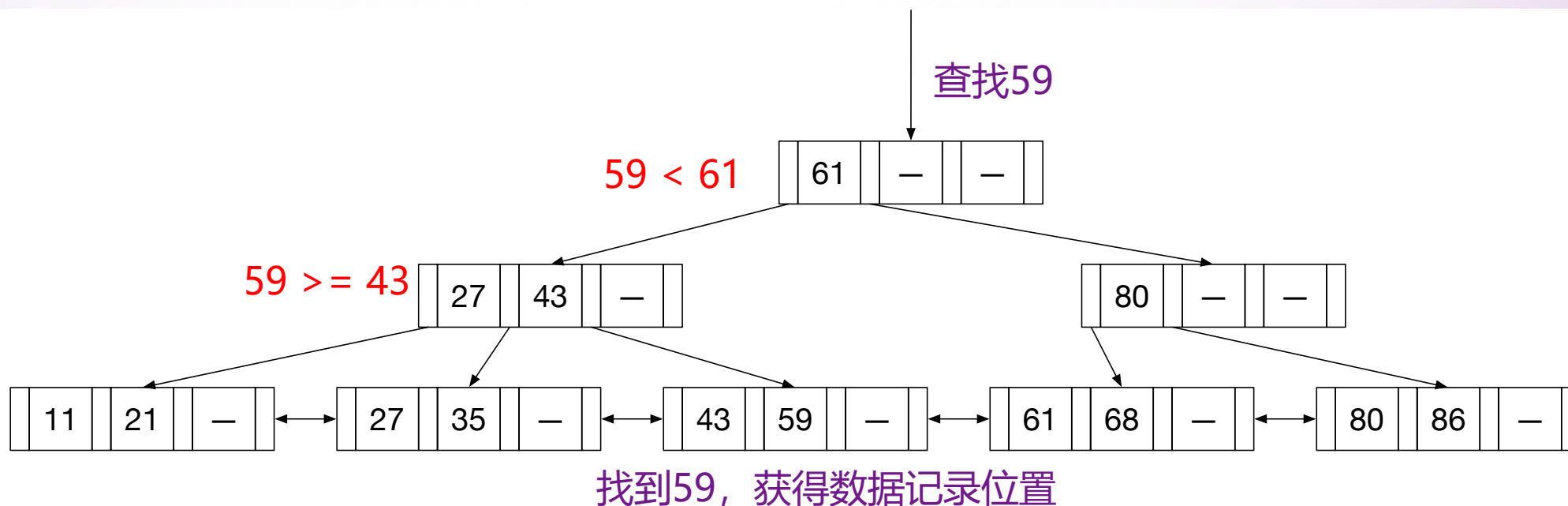
B+树索引

- 索引结构
- 查找算法
- 插入算法
- 删除算法

B+树索引

➤ 点查询

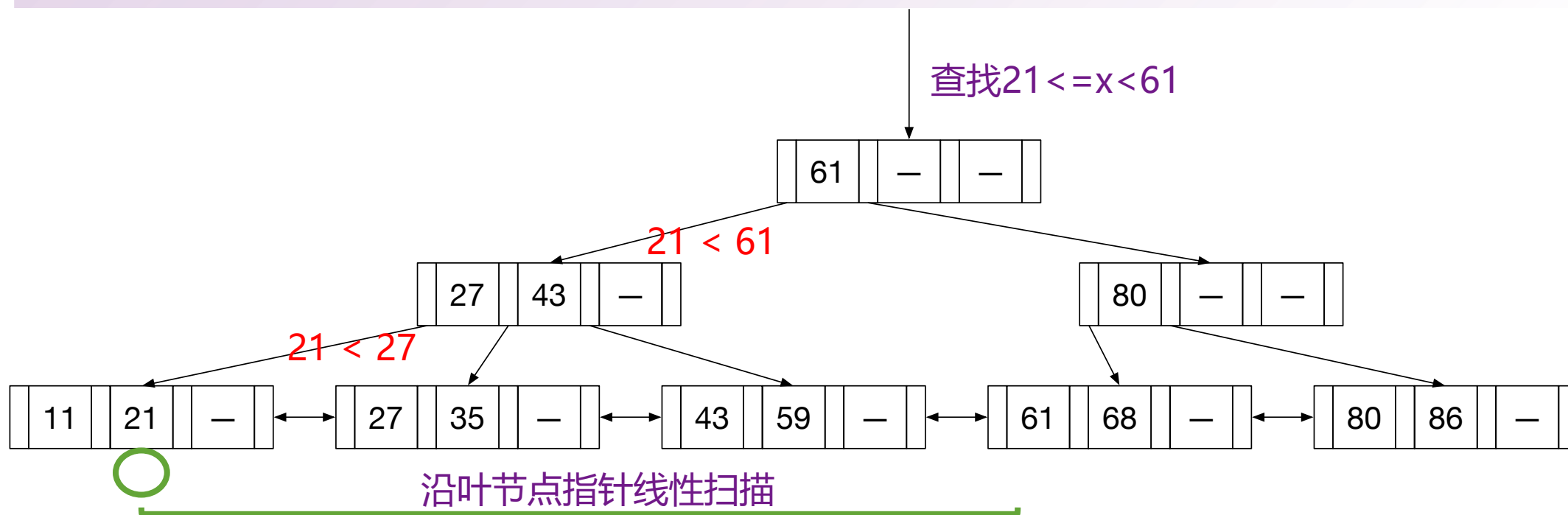
- 从根节点逐层加载索引页面到缓冲区，直到叶子节点
- 注意只判断存在性时，也要访问到叶子节点
- 内部节点存在的键，叶子节点中不一定存在



B+树索引

➤ 区间查询（范围查询）

- 叶节点兄弟之间有指针
- 首先根据查找键查找左端点
- 然后按照节点指针向右线性扫描



B+树索引

- 索引结构
- 查找算法
- **插入算法**
- 删除算法

B+树索引

➤找到应插入的叶子节点 L

➤将索引记录插入到 L 中

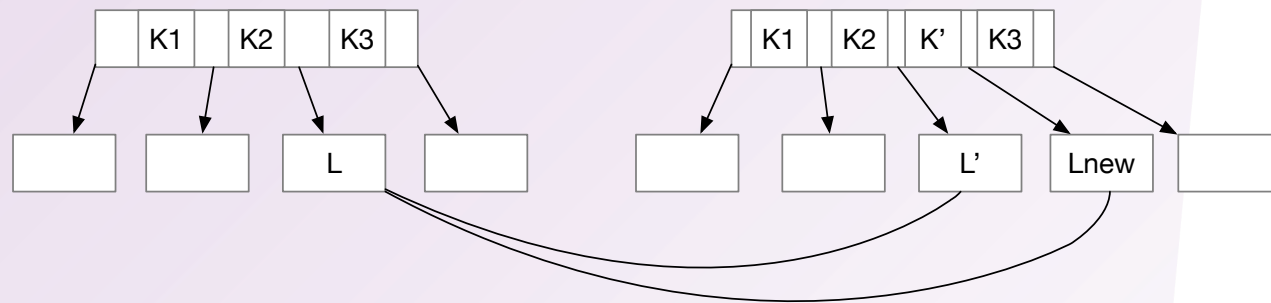
➤如果 L 有多余空间，结束

➤否则，需要分裂 L

- 将节点内容均分为两部分，假设 L 中间键为 K'
- L 中小于 K' 的保存在 L' ；大于等于 K' 的移动到 L_{new}
- 将 K' 插入 L 父节点，并更新指向 L' 和 L_{new} 的指针
- 如果 L 父节点溢出，继续分裂

➤内部节点（非叶结点）分裂：

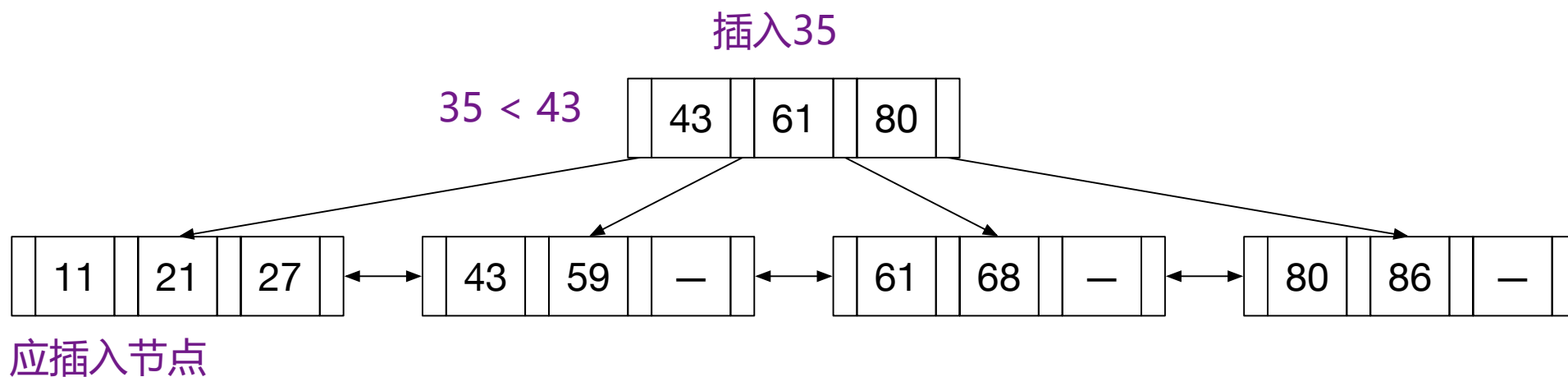
- 中间键 K' 不放在新节点中(L' 和 L_{new})，仅插入父节点



B+树索引

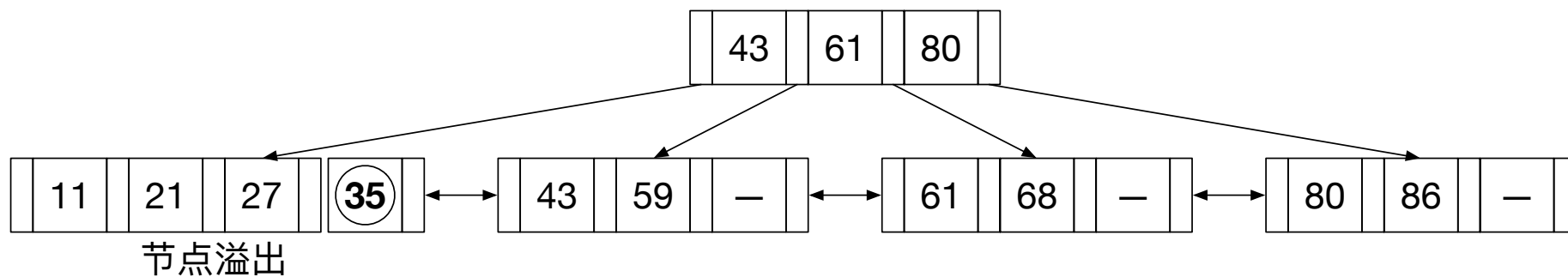
➤ 寻找应插入的节点

- 若叶节点有空余空间，则直接插入



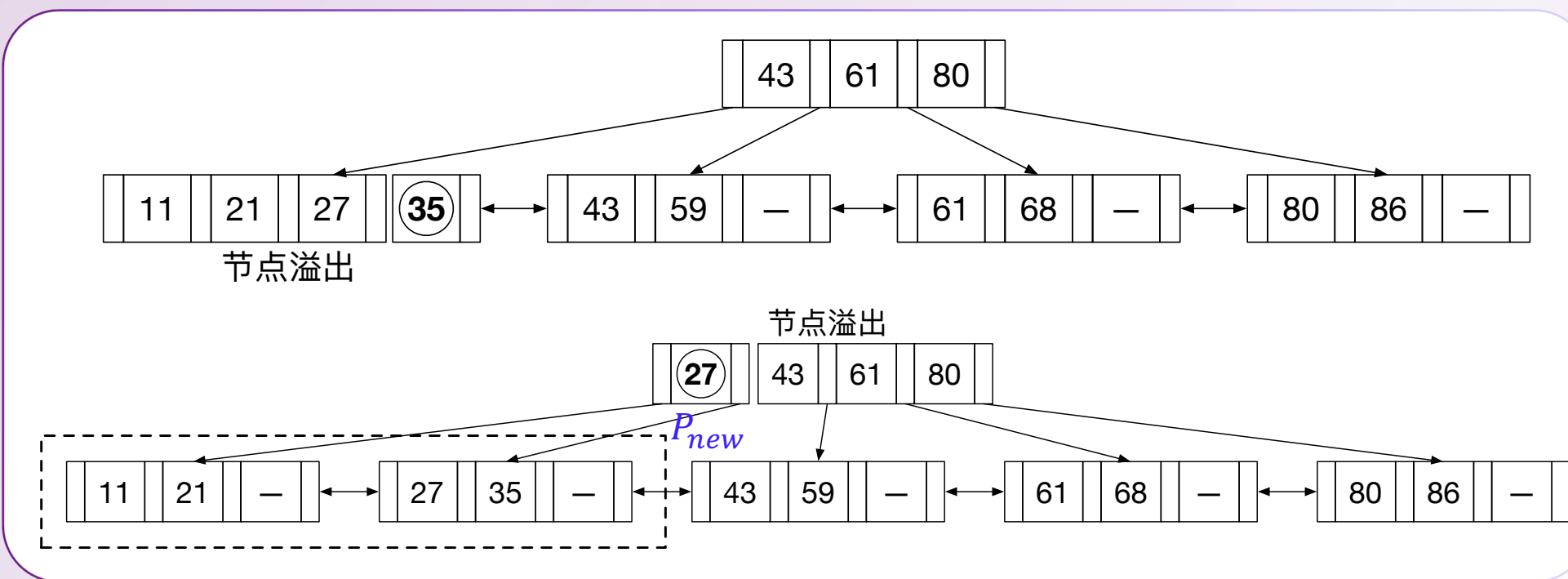
B+树索引

➤ 叶节点溢出，则分裂节点



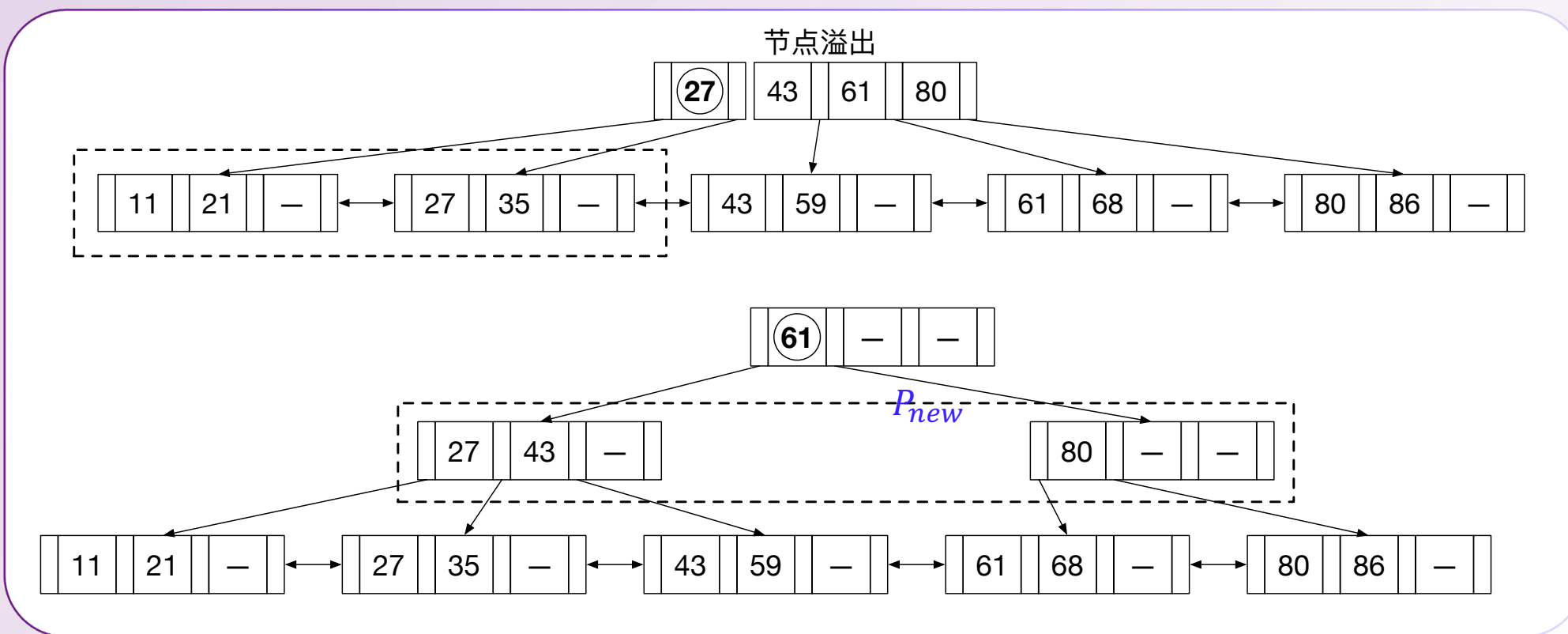
B+树索引

- 叶节点从中间分裂
- $K' = 27$ 向上插入父节点
- 若父节点不溢出，则结束



B+树索引

- 内部节点溢出，从中间分裂， K' 不保留在分裂后的非页节点中
- $K' = 61$ 向上插入父节点（无父节点则分配新根节点）



B+树索引

- 索引结构
- 查找算法
- 插入算法
- 删除算法

B+树索引

➤找到待删除索引记录所在叶子节点

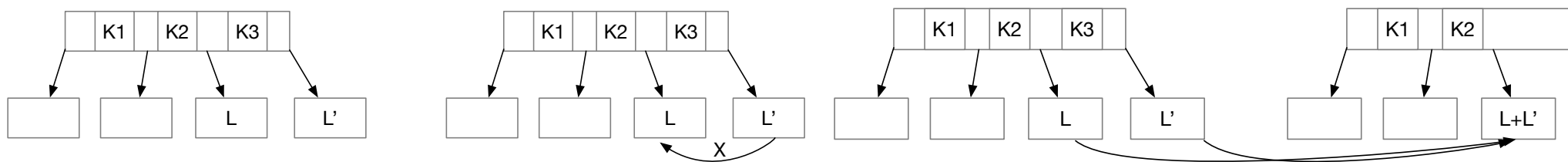
➤从叶子节点中删除索引记录

➤如果节点中键数目大于等于 $\lceil \frac{m}{2} \rceil - 1$,则结束

➤如果节点中键数目小于等于 $\lceil \frac{m}{2} \rceil - 2$, 发生下溢

•尝试从兄弟节点借入索引记录,并更新父节点的键值

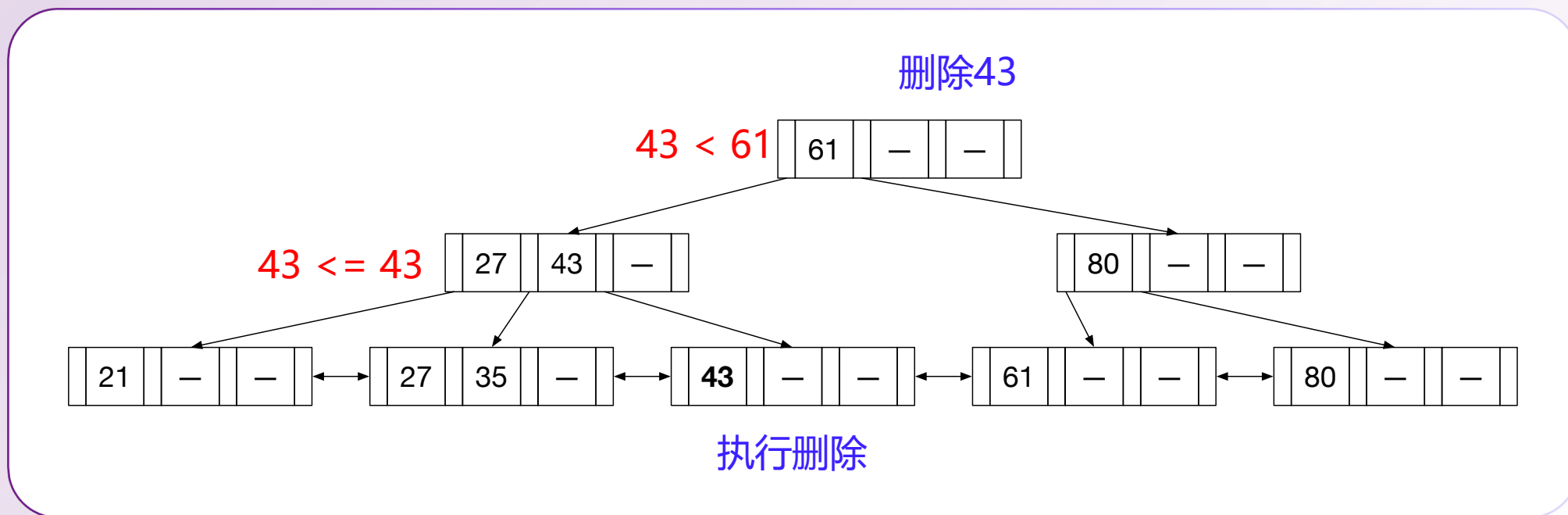
•若兄弟节点无法借入节点, 则与兄弟节点合并, 并从父节点删除兄弟节点的键值和指针



B+树索引

➤ 寻找删除操作发生的叶节点

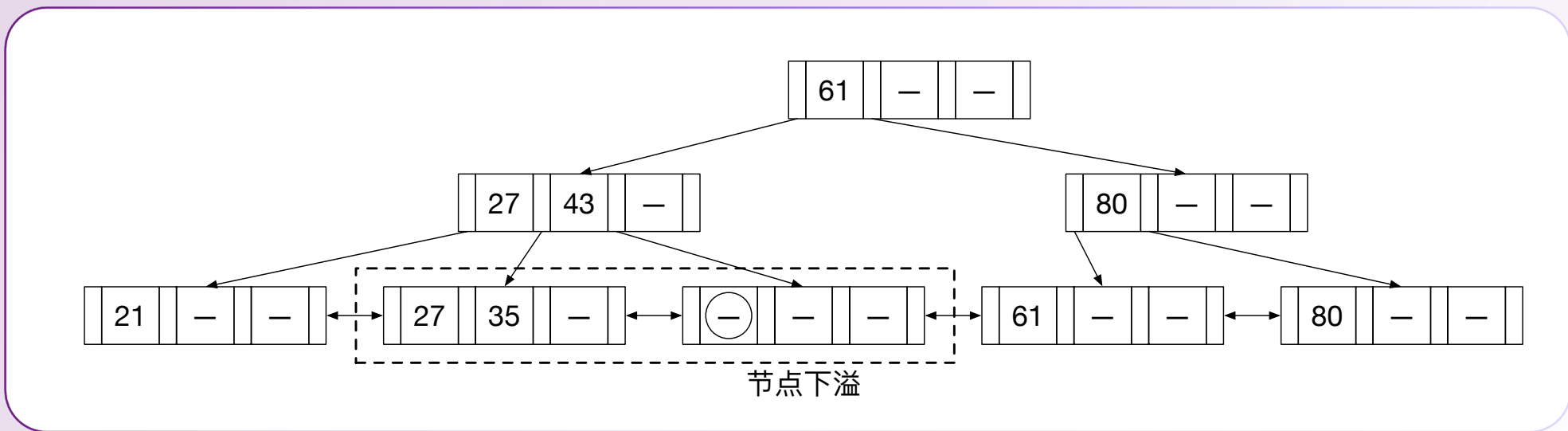
- 如果不存在该查找键，直接结束
- 如果删除操作不会发生下溢，删除索引记录后结束



B+树索引

➤ 如果发生下溢

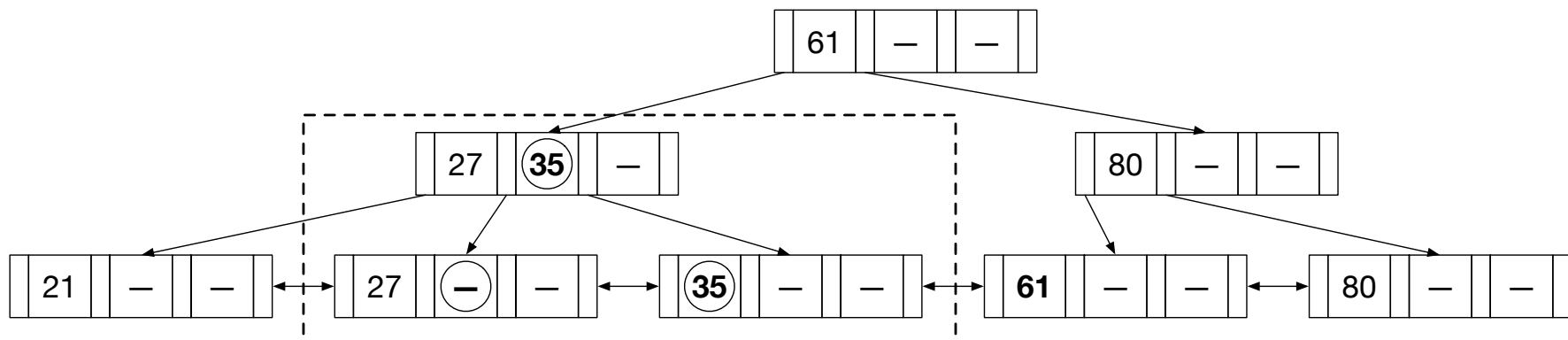
- 尝试从兄弟节点借入索引记录



B+树索引

➤ 如果发生下溢

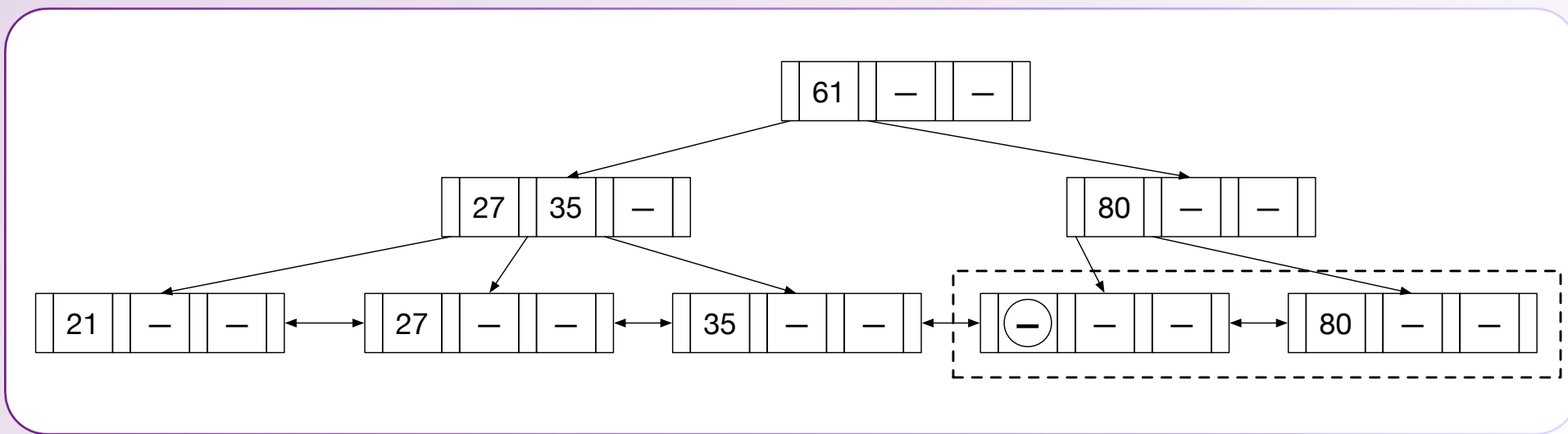
- 尝试从兄弟节点借入索引记录



B+树索引

➤ 如果发生下溢

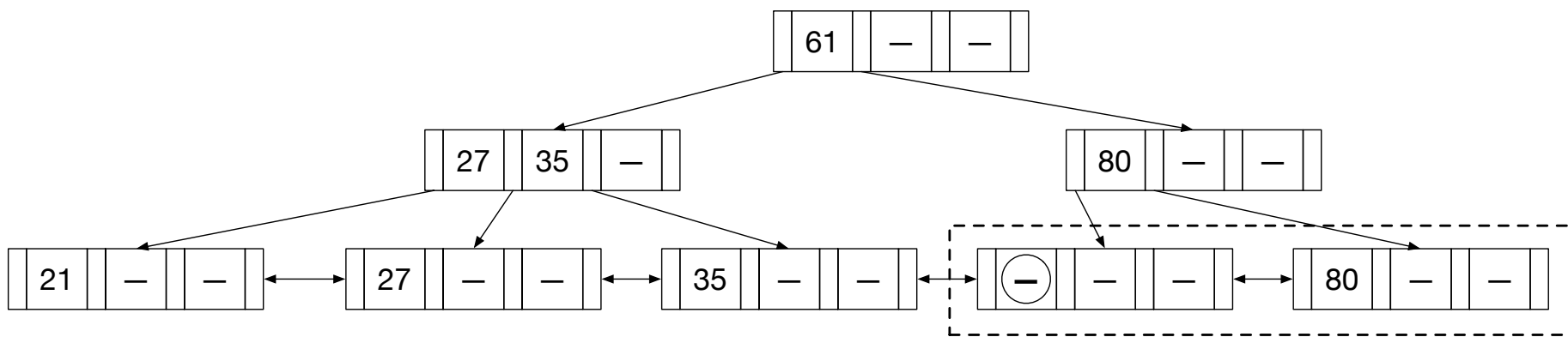
- 尝试从兄弟节点借入索引记录
- 如果兄弟节点无法借入（删除61后），则与兄弟节点合并



B+树索引

➤ 如果发生下溢

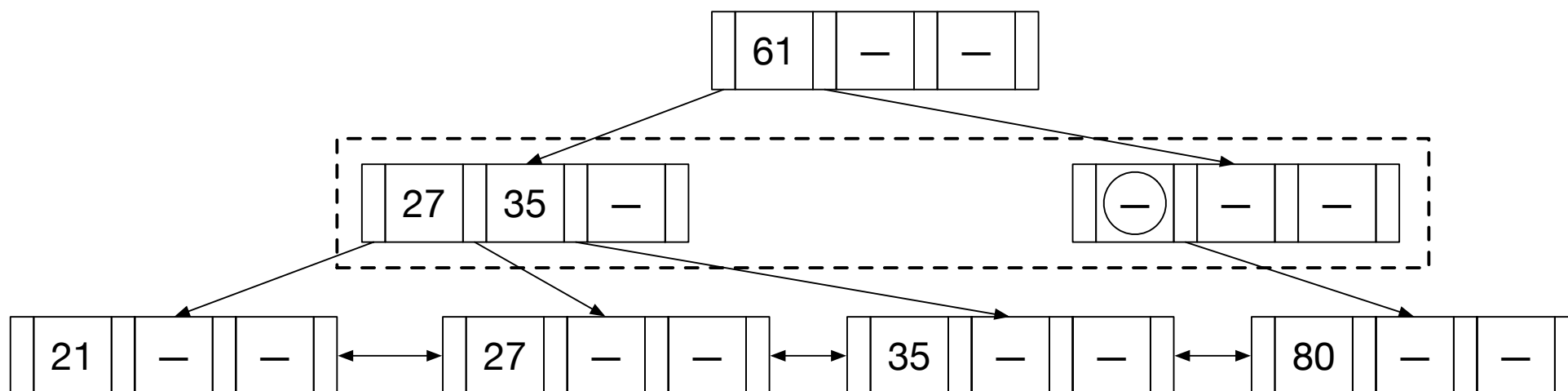
- 尝试从兄弟节点借入索引记录
- 如果兄弟节点无法借入（删除61后），则与兄弟节点合并
 - 从父节点拉取下溢节点与兄弟节点之间的查找键



B+树索引

➤ 如果发生下溢

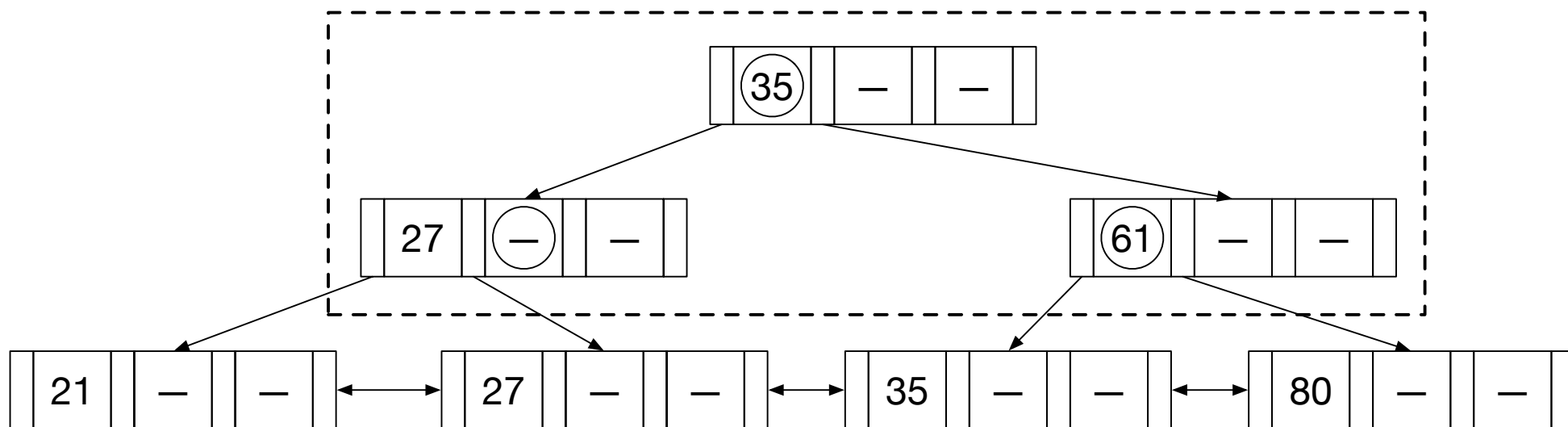
- 尝试从兄弟节点借入索引记录
- 如果兄弟节点无法借入（删除61后），则与兄弟节点合并
 - 从父节点拉取下溢节点与兄弟节点之间的查找键



B+树索引

➤ 父节点发生下溢出，同样方法进行解决

- 此处为从兄弟节点借入节点



B+ 树并发控制

➤ 锁耦合

自上而下：孩子安全后释放祖先锁

乐观自下而上

➤ B-link树

最多锁三个节点

➤ Bw树

无锁结构

原位更新→异步合并更新

B+树索引

□ 事实标准 B+tree (占绝对统治地位时期: 1972-2013)

内存版本 B+tree 通过缓存行对齐, 预取等技术改进后性能优势保持

SIGMOD'00. Making B+trees cache conscious in main memory.

SIGMOD'01. Improving index performance through prefetching.

内存索引

- 内存数据库的索引节点无需映射到磁盘页，仅需存储记录在内存中的地址
- 传统的数据库索引注重于提升磁盘I/O效率，内存数据库在查询更新数据时无需执行磁盘I/O
- 内存数据库索引性能优化目标：
 - 降低索引结构所占的空间，提升索引的内存空间利用率
 - 优化CPU缓存的利用率
 - 适应新型硬件特性

内存索引

- 提升内存空间利用率的索引

降低索引结构所占的空间，使内存索引的结构更加紧凑，提升索引的内存空间利用率。如：T树以及ART树

- 具有缓存感知的内存索引

- 缓存感知：随着硬件技术的发展，CPU与内存数据传输速度差异增大

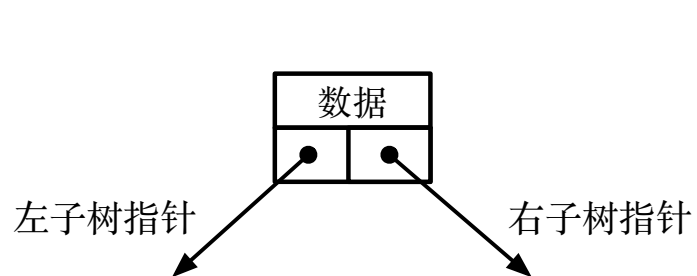
- 内存索引技术的目标为在CPU缓存中尽可能多的存放数据，如CSS树，CSB+树

- 适应新型硬件特性的索引

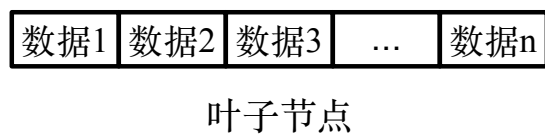
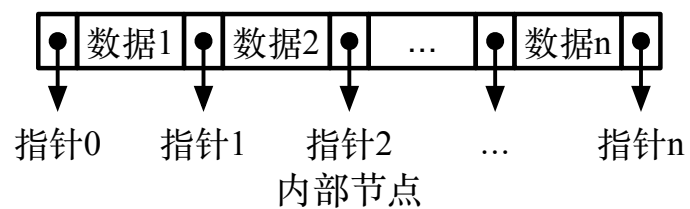
如能够适应多核并行的Mass树等

T 树

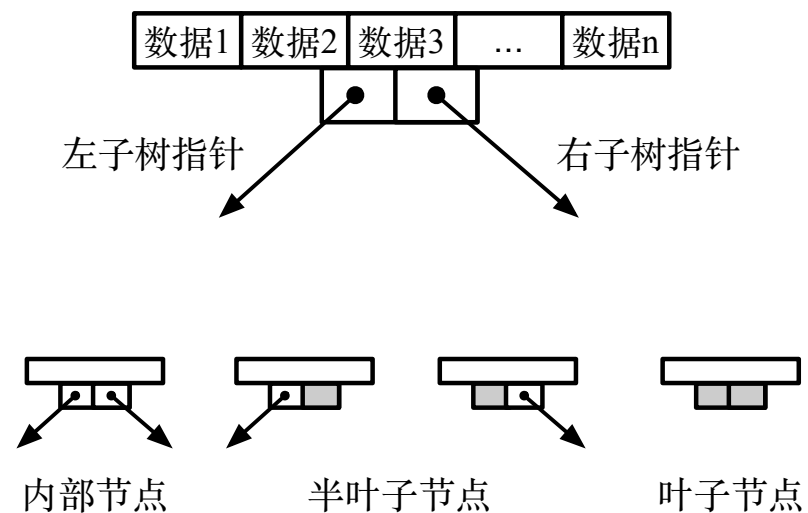
T树包含三种类型的节点：同时具有所有子树的内部节点；仅具有一个子树的半叶节点以及没有子树的叶子节点



(a) AVL树节点的结构



(b) B树节点的结构



(c) T树节点的结构

T 树

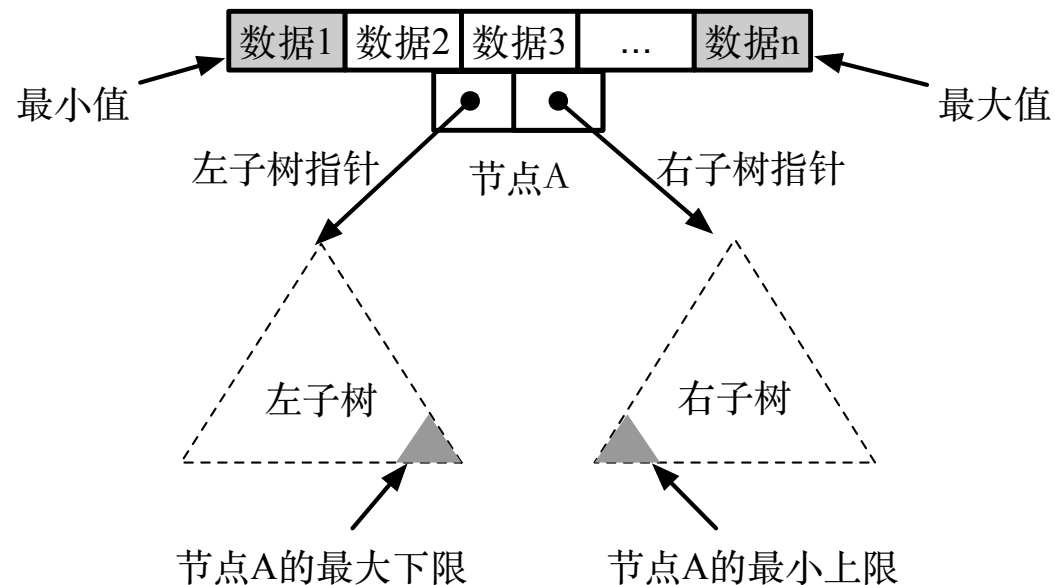
T树每个节点都按顺序存储关键字

节点中所有键的上下限范围：
最小值以及最大值

节点左子树包含的记录的键均小于节点A的最小值，
右子树包含的记录的键均大于节点A的最大值

记录节点所能容纳键的最大数量和最小数量

当前实际存储键的数量



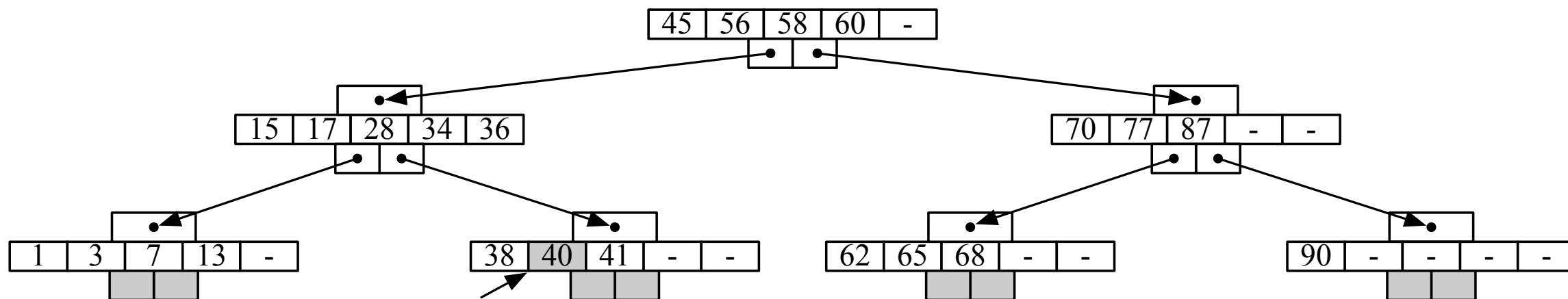
T 树

- T树的查找算法

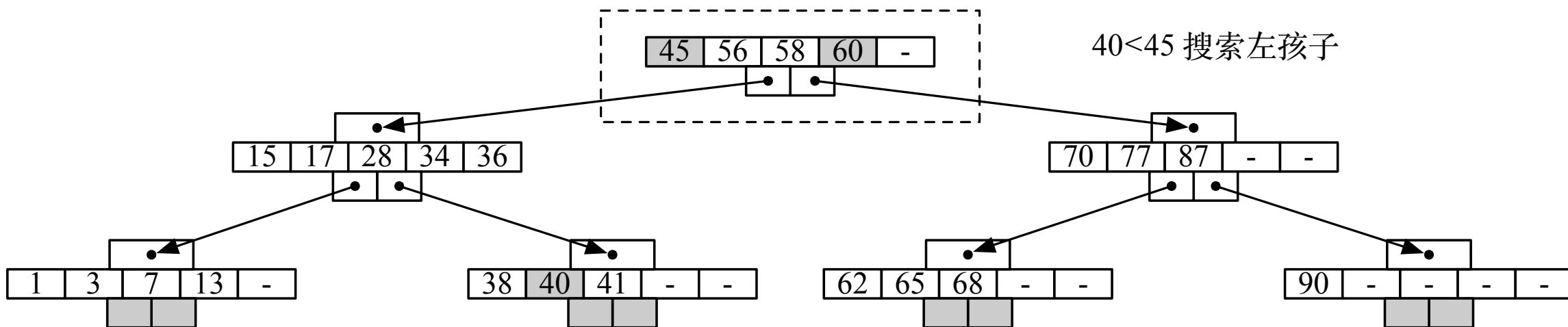
查找算法与B树的查找算法相似

- 1、搜索节点的 $MinK$ 以及 $MaxK$
 - 2、检查目标键值是否在由当前节点的范围
 - 3、若在这一范围：使用二分法进行查找
 - 4、如果小于当前节点的最小值，则搜索当前节点的左孩子节点；
 - 5、如果大于当前节点的最大值，则搜索当前节点的右孩子节点。
- T树的插入以及删除操作均基于查找算法，若插入/删除造成树的不平衡，需要执行旋转操作进行平衡。
 - T树节点的大小是有限制的，若存储值的数量超出技术范围时，树会被重新平衡

T 树

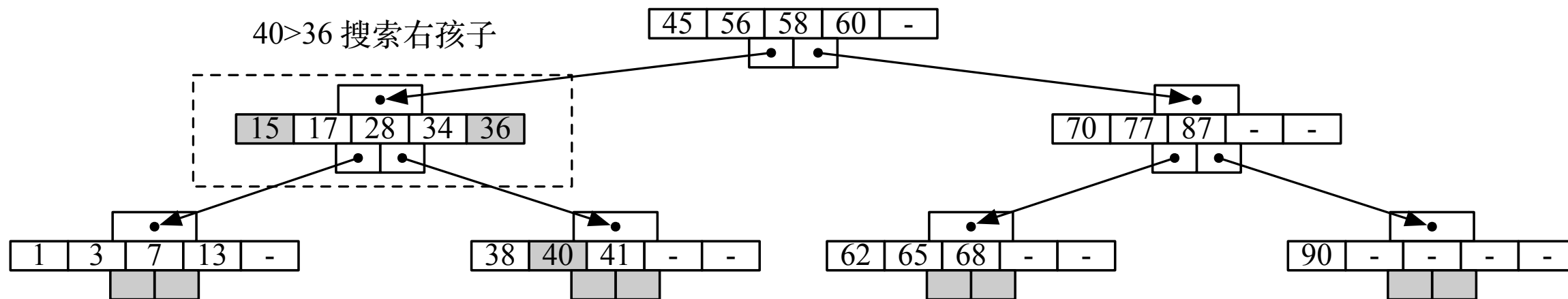


(a)

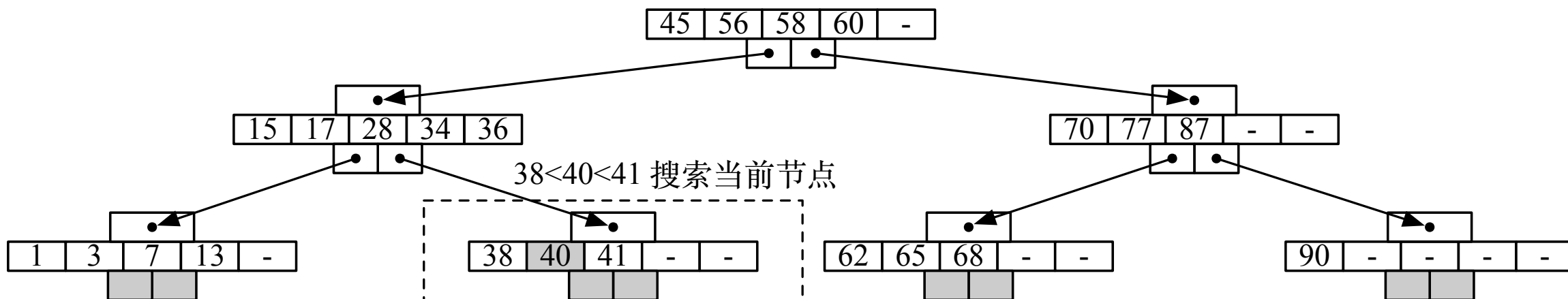


(b)

T 树



(c)



(d)

ART 树

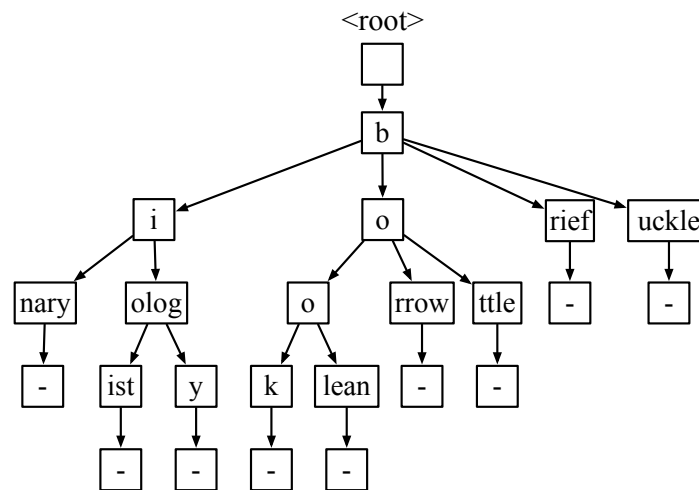
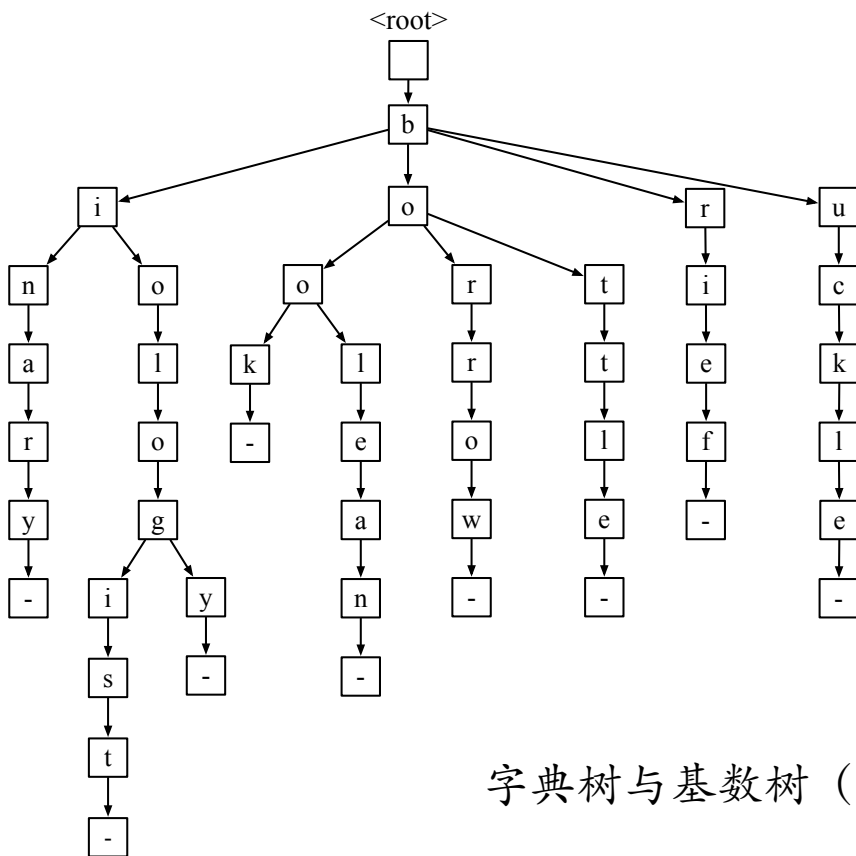
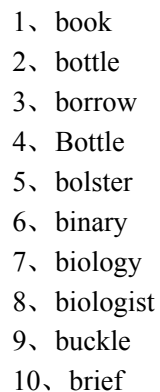
- ❑ 后起之秀 ART (自适应字典树, 从挑战者到站稳脚跟: 2013-2018)
针对内存&缓存特性, 优化字典树结点结构, 创新轻量级并发控制
ICDE'13. The Adaptive Radix Tree: ARTful Indexing for Main-Memory Databases.
ICDE'15. A comparison of adaptive radix trees and hash tables.
DaMoN'16. The ART of practical synchronization.
SIGMOD'18. Building a bw-tree takes more than just buzz words.
Adopted By DuckDB in 2018 (The only one index in the first version)

ART 树

- 自适应基数树 (adaptive radix tree, ART树)
- 一个节点孩子空值比较多, 如何处理?
- 四类节点
 - Node4: 一个key数组(4), 一个child pointer数组(4)
 - key数组存key, child pointer指向孩子
 - 0-4个孩子
 - Node16: 一个key数组(16), 一个child pointer数组(16)
 - key数组存key, child pointer指向孩子
 - 5-15个孩子
 - Node 48 一个key数组(48), 一个child pointer数组(48)
 - 数组下标是key, 数值指向child下标
 - 16-47个孩子
 - Node256 : 一个key数组(256)
 - 数组下标是key, 数值是child指针
 - 48-256个孩子

ART 树

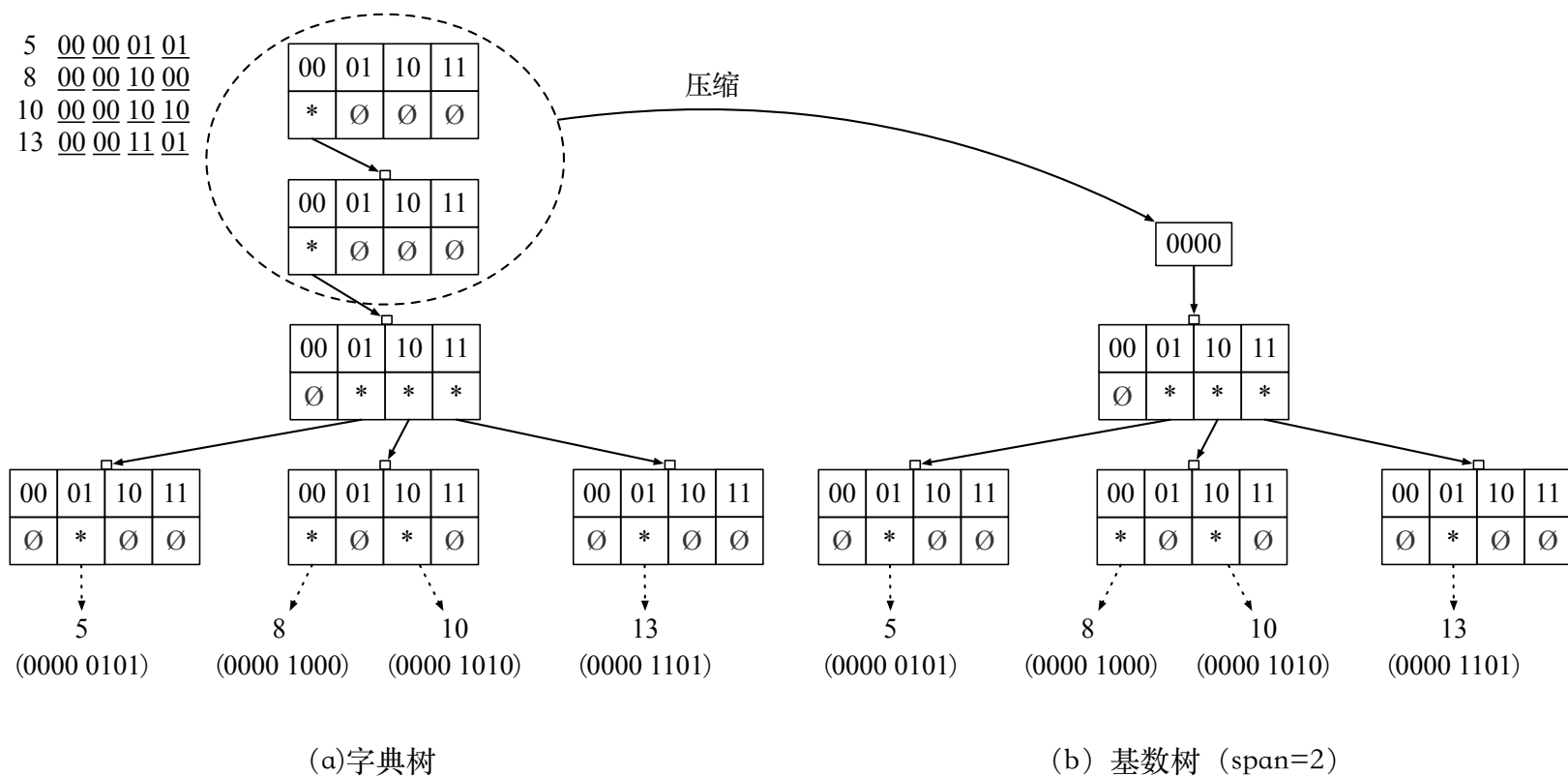
- 自适应基数树 (Adaptive Radix Tree, ART) 基于基数树设计的一种能够调节节点大小的索引结构
- 基数树 (Radix Tree) 是一种压缩前缀树，相较于字典树，基数树能够节省空间



字典树与基数树（存储字符型数据）

ART 树

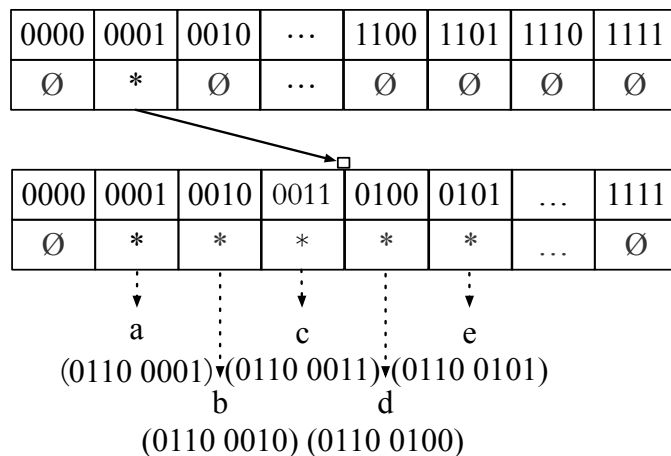
- 自适应基数树 (Adaptive Radix Tree, ART) 基于基数树设计的一种能够调节节点大小的索引结构
- 基数树 (Radix Tree) 是一种压缩前缀树, 相较于字典树, 基数树能够节省空间



ART 树

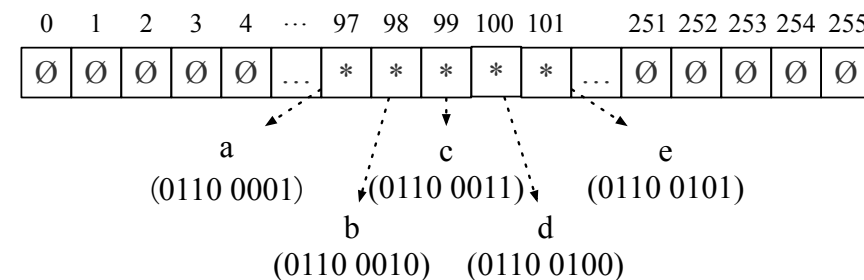
- 在基数树中，一个存储 k 个键的基数树的高度为 $\lceil \frac{k}{s} \rceil$ ，随着跨度增加，树的高度降低
- 当跨度为4时，基数树的高度为2，当跨度为8时，树高为1

a 0110 0001
b 0110 0010
c 0110 0011
d 0110 0100
e 0110 0101



(a) 基数树 (span=4)

a 0110 0001
b 0110 0010
c 0110 0011
d 0110 0100
e 0110 0101



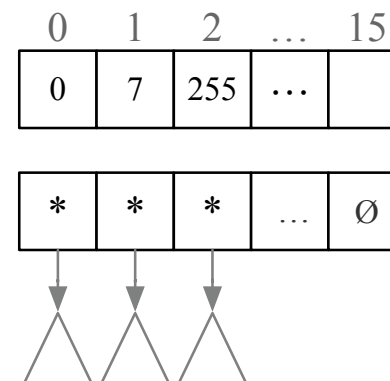
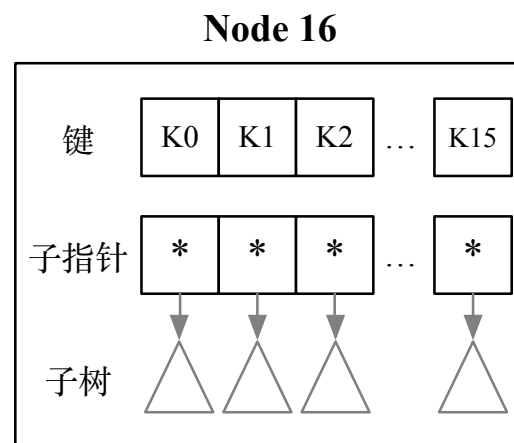
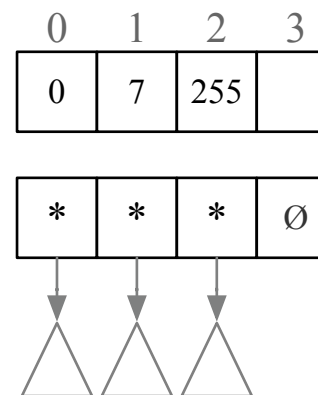
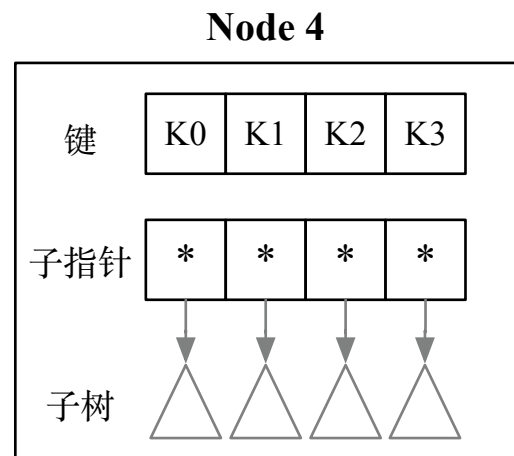
(b) 基数树 (span=8)

ART 树

- 与传统的基数树相比，ART树中每个节点可以容纳的键的数量可以动态变化的
- 这种自适应的节点不会影响树的结构（树的高度），仅会影响每一个节点的大小。

ART 树

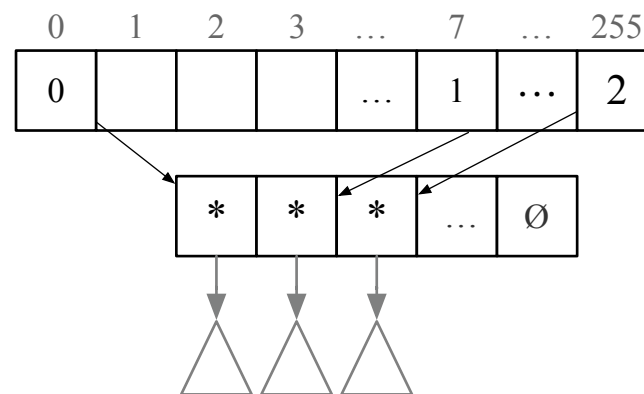
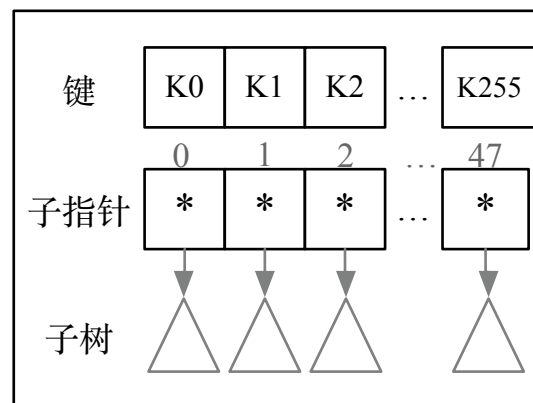
- 内部节点可以按照前缀的最大容量分为4个类型



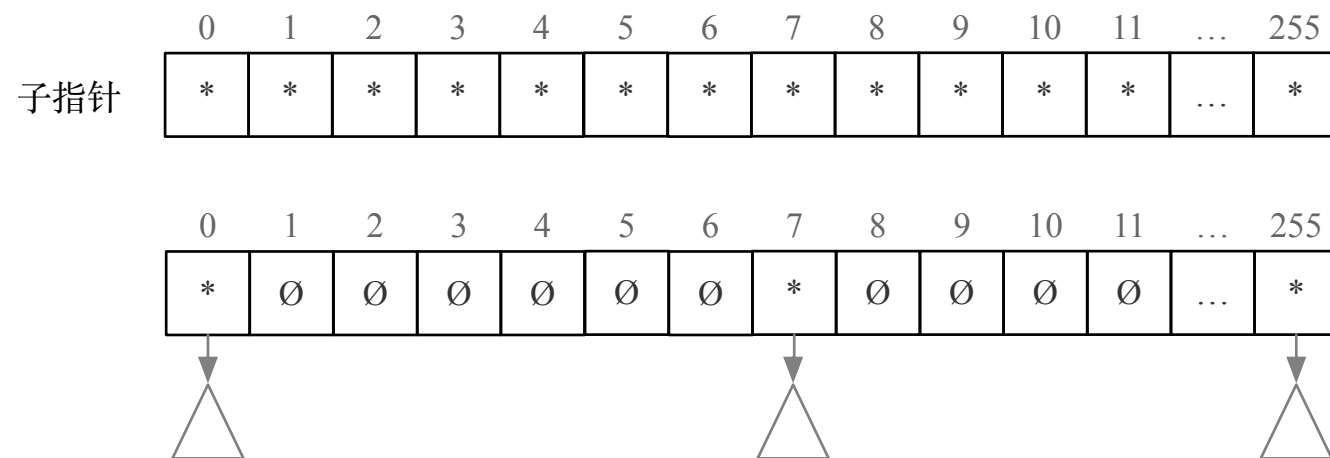
ART 树

- 内部节点可以按照前缀的最大容量分为4个类型

Node 48



Node 256



ART 树

- **ART树的查找算法**

ART树通过使用包含键连续字节的数组来遍历树，直到遇到一个叶子节点或空指针。

如果当前节点为叶子节点，则进行完整匹配，匹配成功则查找完成；

若当前节点为内部节点，需要对前缀进行匹配，成功则继续在子节点中寻找。

- 四种内部节点中的查找：

- (1) Node4节点：使用循环进行查找。

- (2) Node16:使用SIMD指令，即使用一条指令并行地比较16个键，或者二分查找进行查找。

- (3) Node48:如果childindex数组存在，使用该数组的下标寻找。

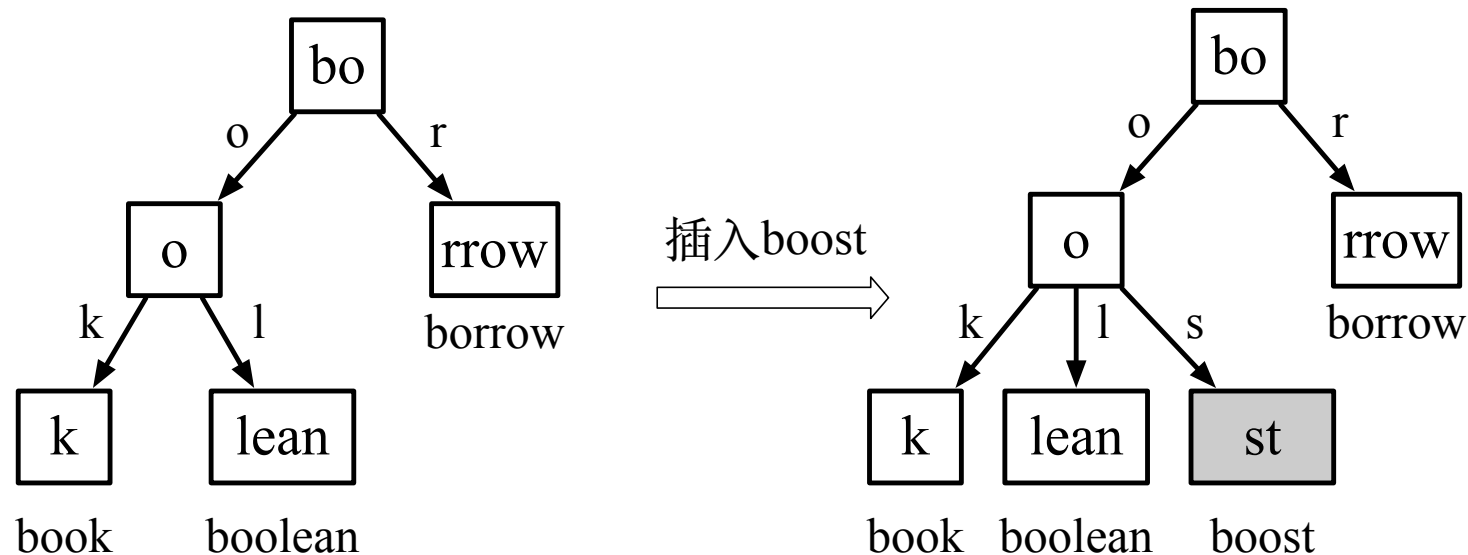
- (4) Node256:直接使用byte作为下标寻找子节点。

ART 树

• ART树的插入算法

ART树的插入算法为从根节点向下查找，不断比较公共前缀，若能够找到叶子节点则更新值，不能找到则生成新的叶子节点

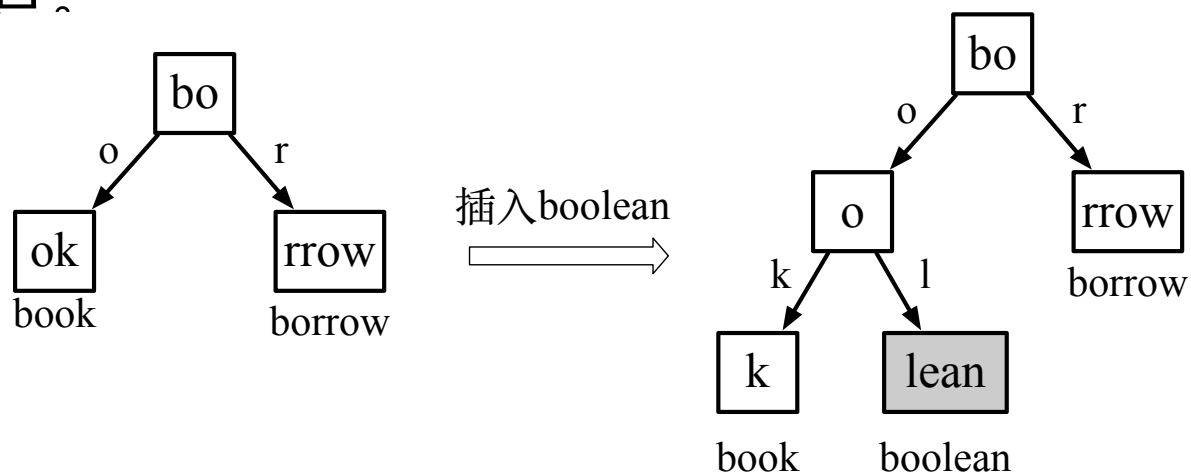
- (1) 首先对树进行遍历，找到待插入的位置。
- (2) 若当前节点为空节点，则插入一个新的叶子节点替换该空节点。



ART 树

• ART树的插入算法

(3) 若当前节点为叶子节点，则将叶子节点分裂为一个包含两个叶子节点的内部节点。

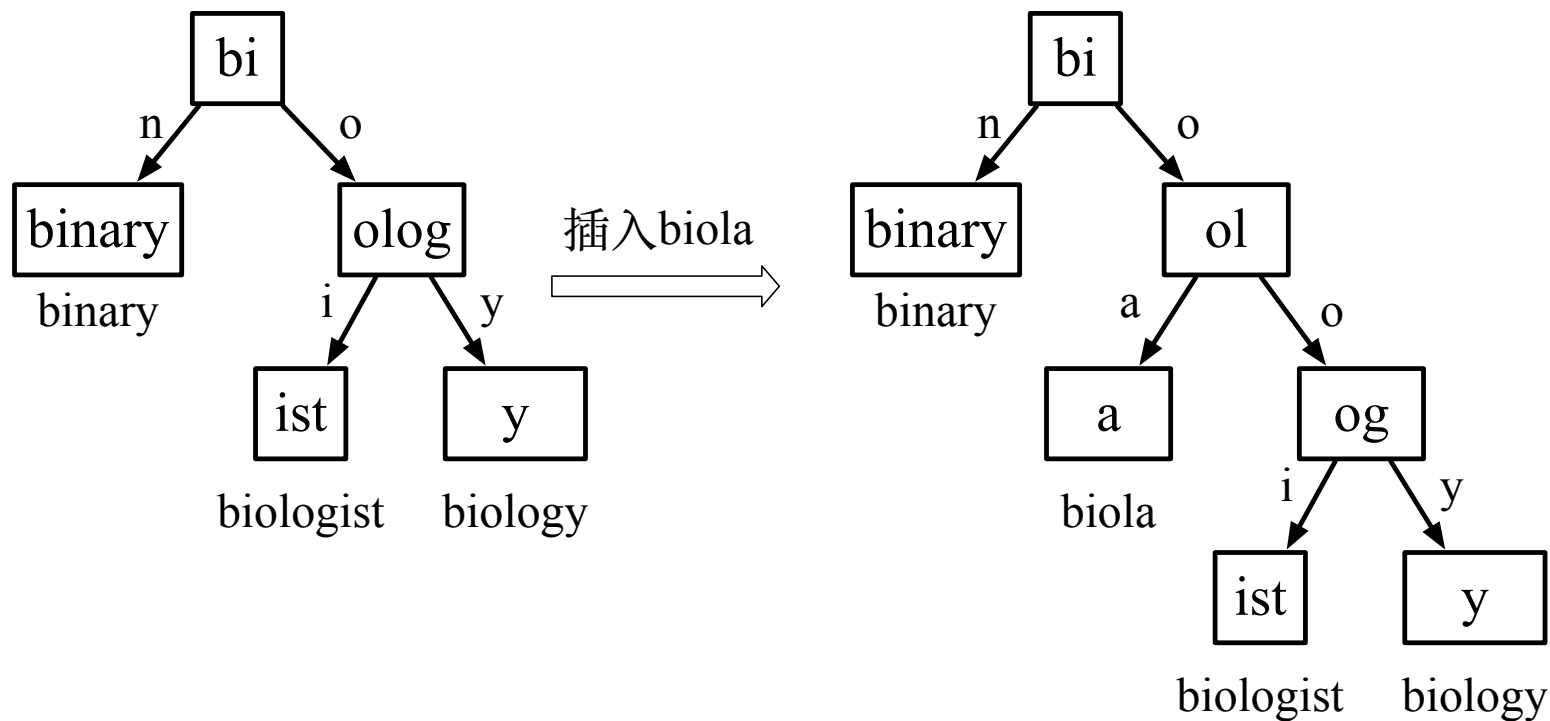


(4) 若当前节点为内部节点，且前缀匹配成功，则寻找其子节点，若子节点存在，则转入下一层，若子节点不存在，则插入子节点，操作与 (2) 中的操作相似，并于执行插入操作前判断是否需要进行节点扩容。

ART 树

- ART树的插入算法

(5) 若当前节点为内部节点，但前缀匹配失败，则需修剪前缀并生成新的内部节点。



- **ART树的删除算法**

删除操作为插入操作的镜像操作。

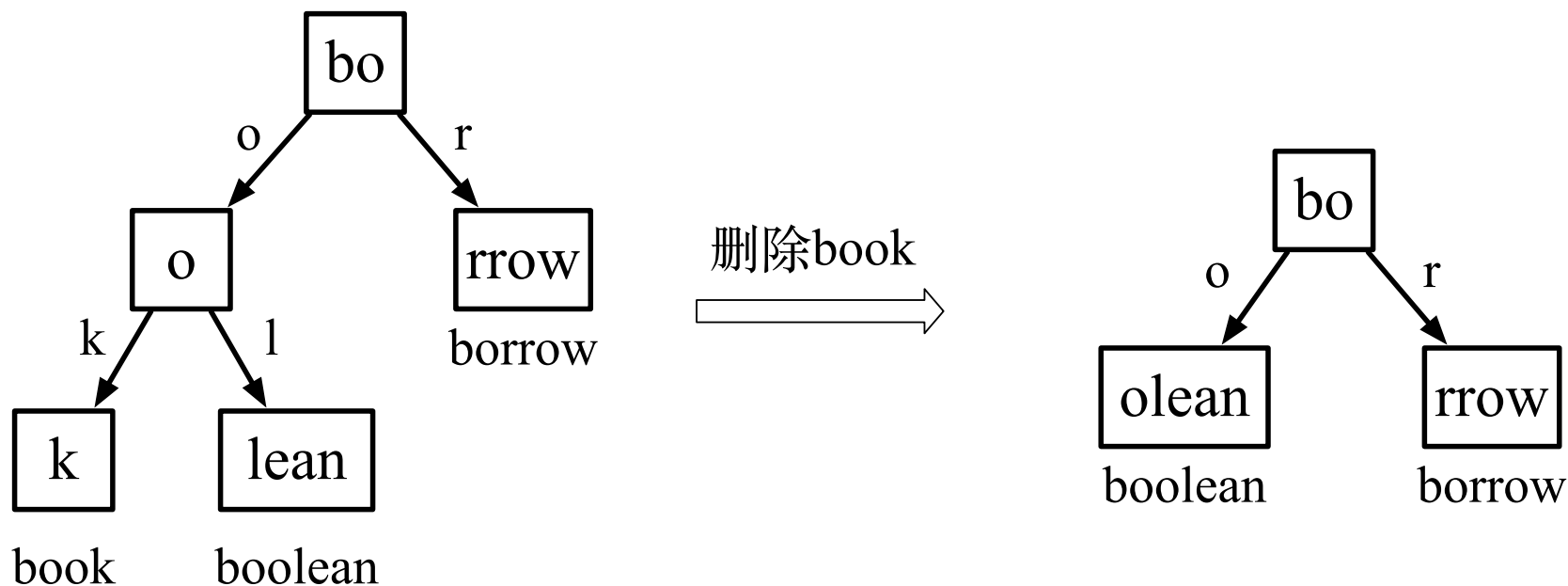
在执行插入操作时，节点可能会发生膨胀，需要修剪前缀，而在执行删除操作时，节点可能会收缩，需要压缩前缀。

ART树的删除操作流程为从根节点向下查找，不断比较公共前缀，若能够找到叶子节点则删除。

ART 树

• ART树的删除算法

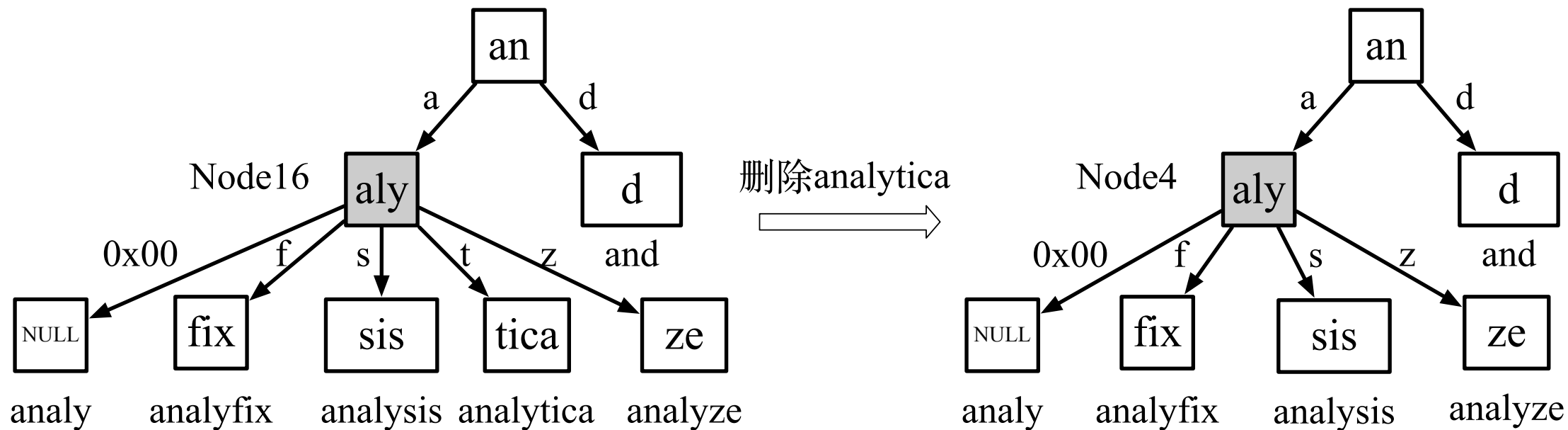
- (1) 首先对树进行遍历，找到待删除的叶子节点位置。
- (2) 删除叶子节点，并压缩前缀。若当前节点为根节点，则删除根节点，并释放内存。



ART 树

- ART树的删除算法

(3) 若删除操作引起内部节点的收缩，需对内部节点进行降级



缓存

缓存（Cache）是一种能够协调CPU与内存数据传输速度差异的结构

内存中的数据可提前备存储至缓存中

若访问的数据已被存储至缓存中，则为缓存命中（CPU需1~2纳秒从缓存完成读取），反之，为未命中（CPU需50~100纳秒从内存完成读取）

缓存命中能够有效缩短CPU获取到数据的时间，提升处理效率，减少CPU访问主存的次数。

缓存行（cache-line）是缓存与内存间数据传输的基本单位，具有缓存感知能力的索引即能够优化缓存行的利用率，提升CPU缓存效率的索引。

CSS树

- CSS树是一种面向OLAP负载的内存索引，它的优化重心在利用缓存行优化索引的搜索时间以及存储索引所需要的空间

- 第0-15号节点被称为内部节点（internal nodes）

存放索引项

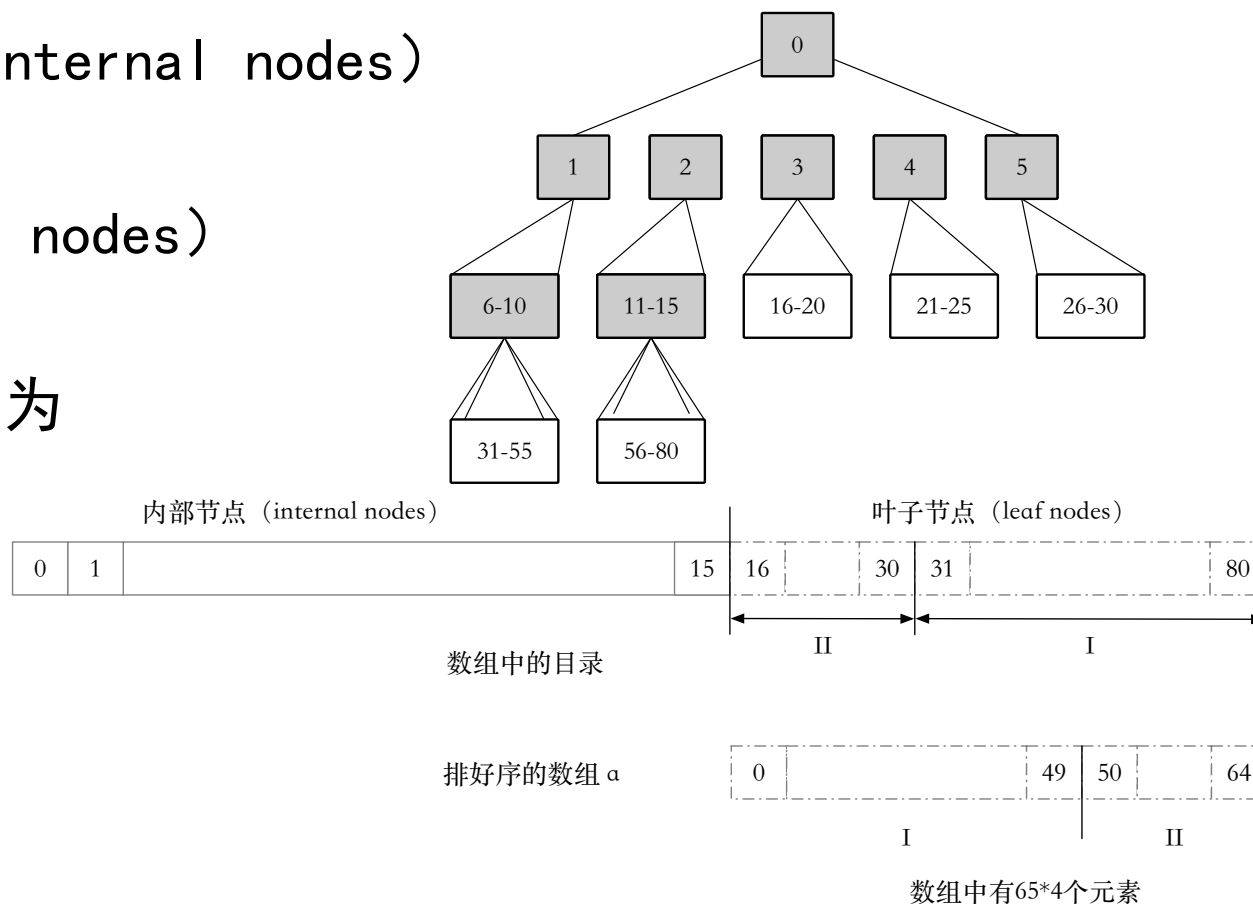
第16-80号节点为叶子结点（leaf nodes）

存放有序数组a中的数据

编号为 b 的节点，其子节点的编号为

$b(m+1)+1 \sim b(m+1)+(m+1)$

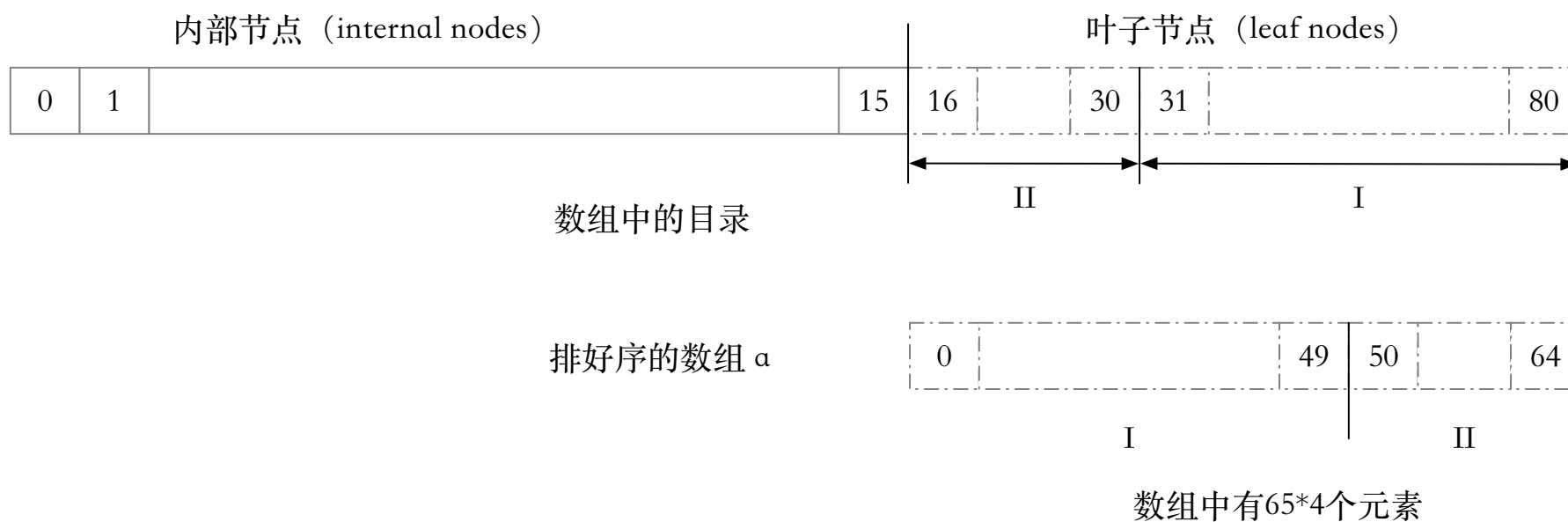
每个节点包含 m 个键，目录数组中
键号 i 对应的节点号为 $\lfloor i/m \rfloor$



CSS树

在数组目录中，节点16-30（II）的存储位置位于节点31-80（I）之前

在有序数组中，叶子节点31-80（I）存储的数据位于叶子节点16-30（II）之前



- 为了能够有效地构建以及搜索CSS树，需要用到一下参数：
 - (1) $N=B*m$ ，为排序数组a的元素个数，B为叶子结点的个数。
 - (2) CSS树中内部节点的数量为 $\frac{(m+1)^k-1}{m} - \left\lfloor \frac{(m+1)^k-B}{m} \right\rfloor$
 - (3) 最底层的第一个叶子结点的编号为 $\frac{(m+1)^k-1}{m}$
 - (4) k的大小为 $\lceil \log_{m+1}(B) \rceil$

- **CSS树的构建**

- (1) CSS树中，每个索引项的值都是该索引项左子树中数据项的最大值。
- (2) 利用循环操作在左子树中不断获取位于最右侧的孩子，达到叶子节点时，最右的孩子即为最大值。
- (3) 索引树的构建顺序为，从最后一个索引项开始往前构建，直到到达根节点。
- (4) 由于最后一个索引项位于最后一个内部节点中，因此需要在开始时计算出内部节点个数。

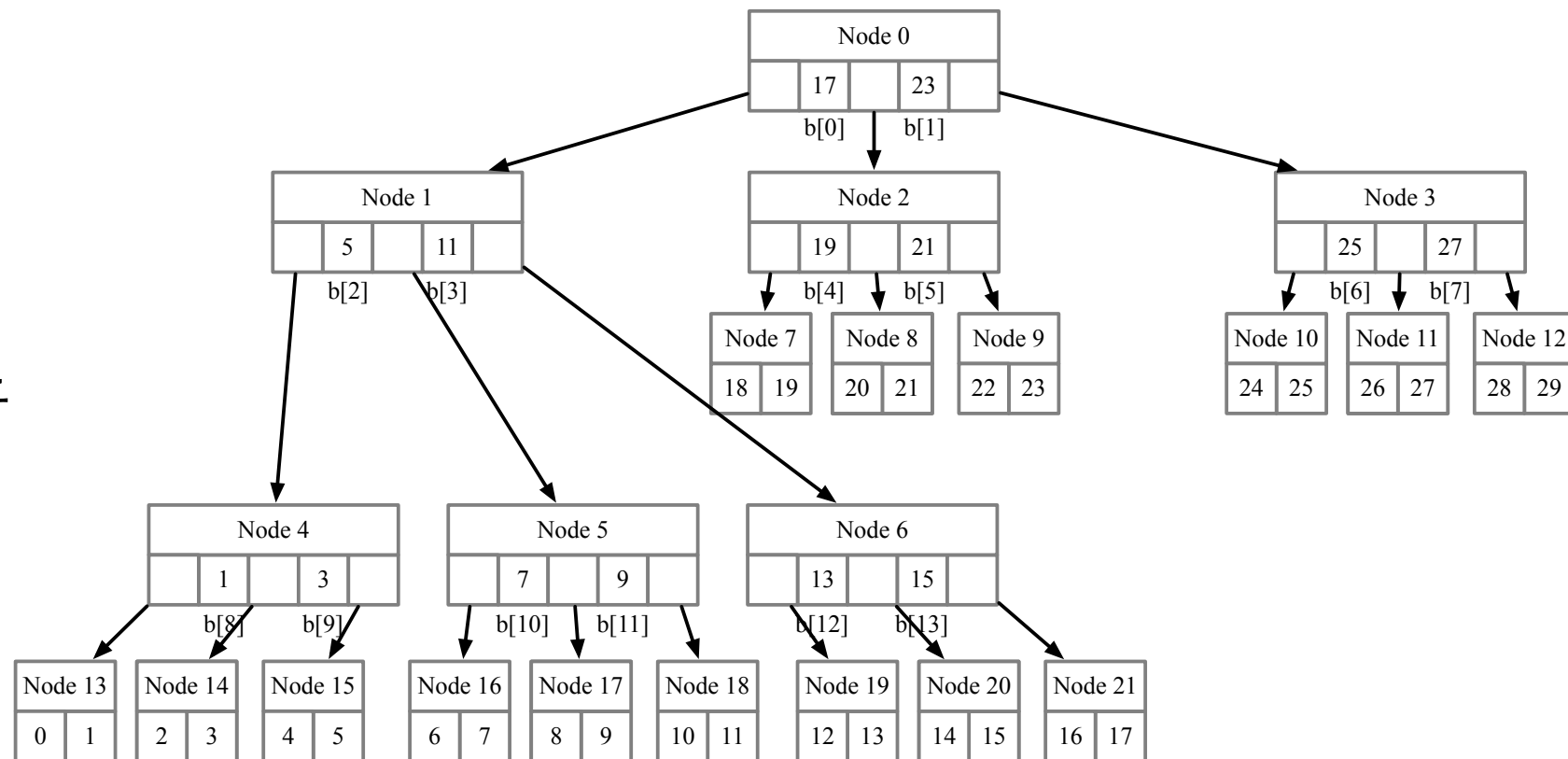
- **CSS树的搜索**

CSS树的搜索与二叉搜索相似

- (1) 在每棵树的内部节点中对节点进行索引项的二分查找，找到接下来要走的分支，直至到达叶子结点。
- (2) 使用二叉查找查看叶子结点中是否包含搜索的数据。

CSS树

- 以有序数组a为例，构建一棵 $m=2$ 的CSS树
- (1) 叶子节点数15个
- (2) 内部节点的数量7个
- (3) 最底层的第一个叶子结点的编号13
- (4) k 的大小为3



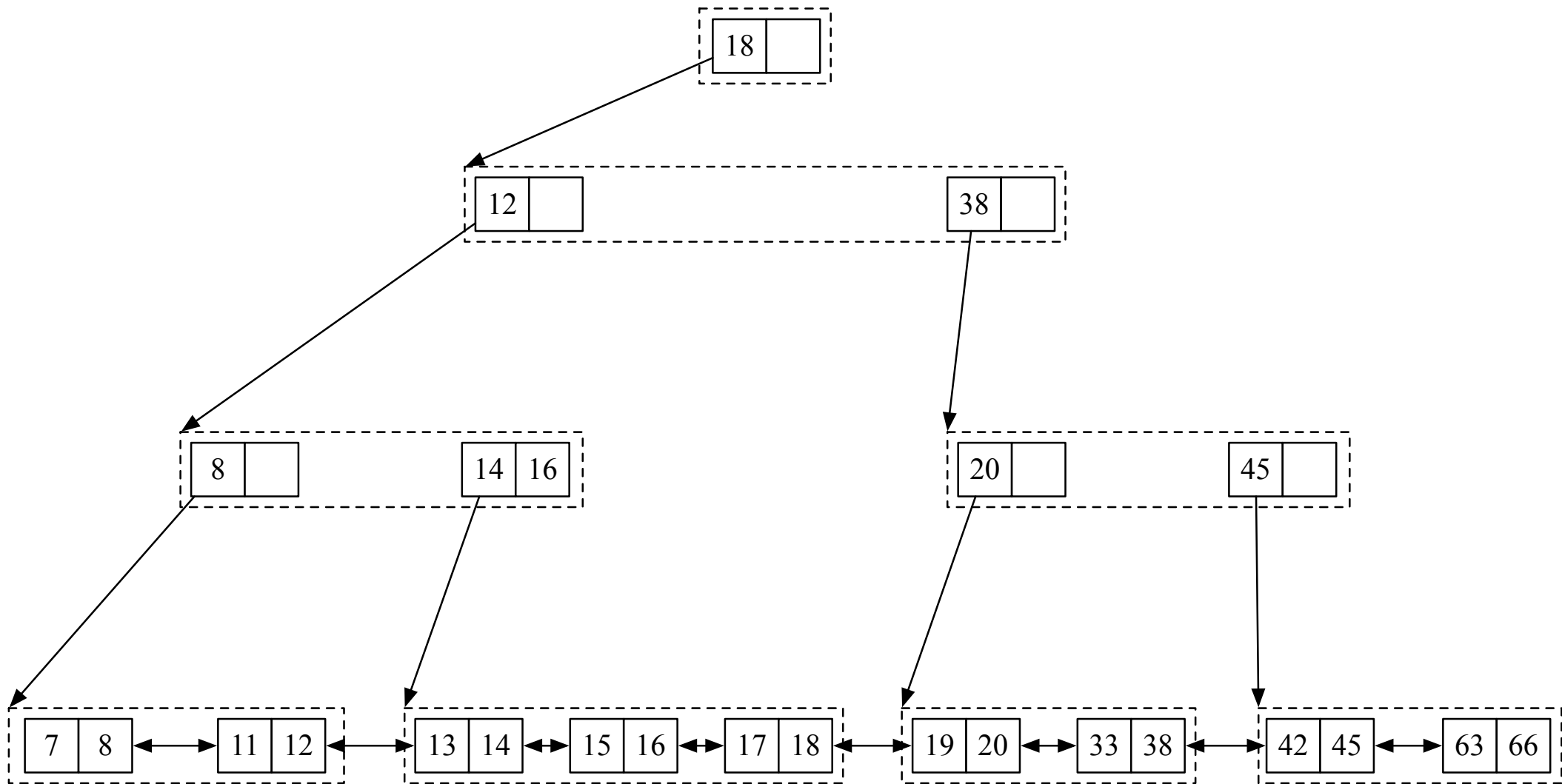
有序数组 a: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29

索引数组 b: 17 23 5 11 19 21 25 27 1 3 7 9 13 15

CSB树

- CSB+树是一种基于B+树的平衡多路搜索树，它能够兼顾优秀的缓存感知能力以及支持增量更新的索引结构。
- 在CSB+树中，节点的子节点被连续地存储在一段地址中，且仅保留指向第一个子节点的指针。
- 与CSS树相似，CSB+树利用offset来查找给定节点的子节点，通过在第一个子节点的地址加上offset，即可查找到其余子节点。

CSB树



- CSB+树的批量加载

CSB+树使用逐级建立索引的方式实现索引结构的批量加载。

- (1) 计算所有叶子节点所需的空間，并计算出上一级需要的内部节点的数量及需要分配的连续空间。
- (2) 使用叶子节点的最大值来填充上一级的内部节点，并保存指向第一个子节点的指针。
- (3) 重复这一过程直至仅剩一个高层级节点，此节点即为根节点。

- CSB+树的查找算法

CSB+树的查找算法与B+树相似

- (1) 需获得小于搜索键的最大键值 K
- (2) 并将 K 的 $offset$ 加到第一个子节点的指针上，即可获得目标子节点的地址。如 K 为节点中的第 n 个键，可以通过 $child = firstChild + n$ 计算出。

- CSB+树的插入算法

CSB+树的插入算法与B+树相似，首先使用查找算法找到待插入的位置。

- (1) 若待插入的叶子节点有足够空间容纳新的索引记录，则直接插入新的键。
- (2) 若叶子节点空间不足，插入新的记录会导致节点溢出，需要分割叶子节点。
- (3) 根据父节点是否有足够空间容纳新的键，可以分为两种情况：

CSB树

- CSB+树的插入算法

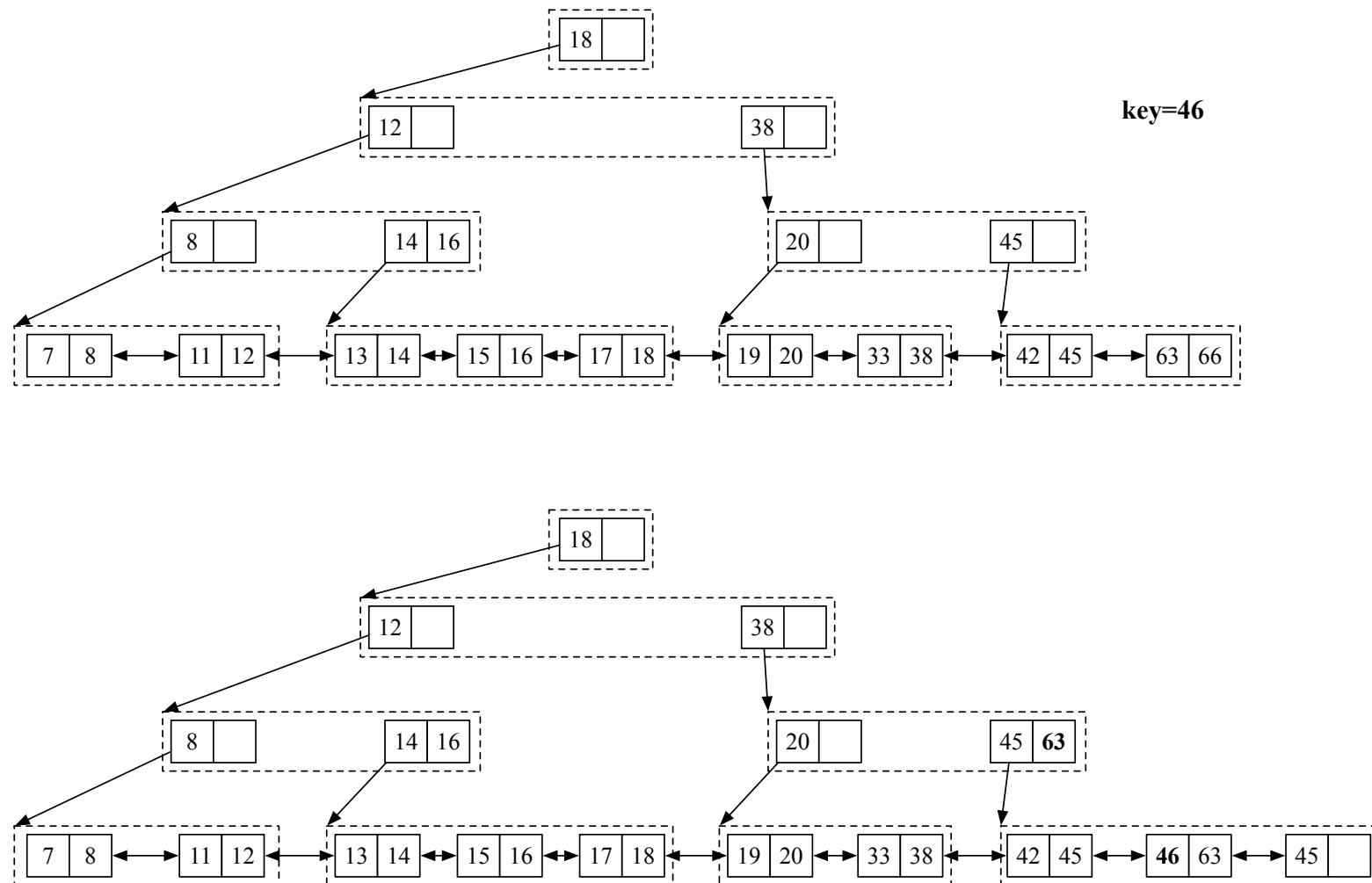
若父节点 p 有足够空间

1、则需创建新的比原节点组 g 多一个节点的节点组 g_0 。

2、复制 g 的所有节点至 g_0 ，在 g_0 中保存被分割的两个节点。

3、更新父节点 p 指向第一个子节点的指针 f ，更新后的 f 指向 g_0 。

4、释放节点组 g 。



CSB树

- CSB+树的插入算法

若父节点 p 空间已满，且插入新的索引记录会导致父节点发生溢出

- 1、需创建新的节点组 g_0

- 2、将原节点组 g 中的节点平均分配在 g 与 g_0 中

- 3、创建新的节点 p_0

- 4、将父节点 p 的一半键放置在 p_0 中， p_0 指向的第一个子节点的指针指向 g_0 。

- 5、若分割 p 会导致其父节点溢出，递归地执行分割操作

Limitations of B+tree

- **Fixed Structure:** Hierarchical nodes require multiple levels of traversal ($\log(n)$ comparisons), leading to higher CPU and memory costs.
- **Memory Overhead:** Each node holds pointers, using significant memory for large datasets.
- **Rigid Partitioning:** B+ Trees don't adapt to predictable key patterns, which can cause unnecessary disk I/O and slower access.

B+tree vs. Learned Index

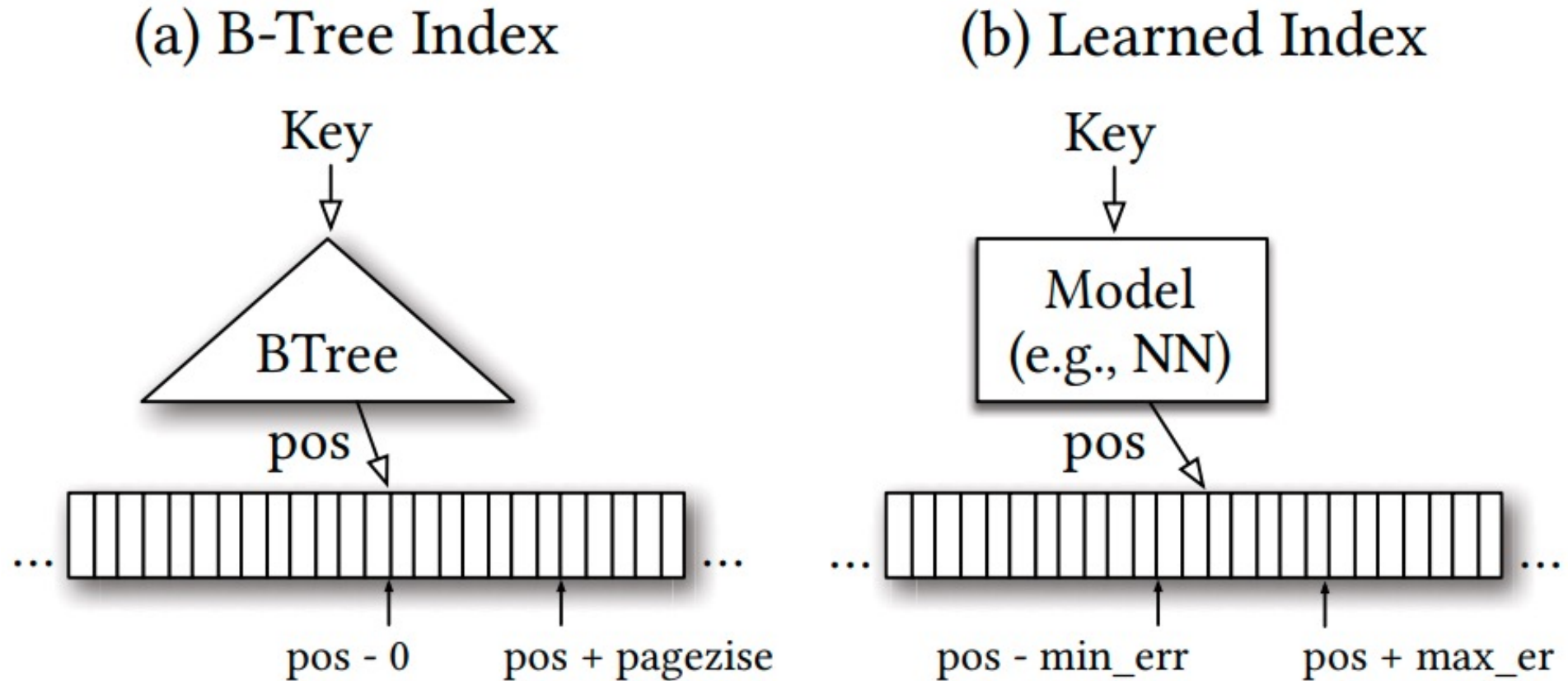


Figure 1: Why B-Trees are models

Kraska T, Beutel A, Chi E H, et al. The case for learned index structures[C]//Proceedings of the 2018 international conference on management of data. 2018: 489-504.

Learned Index

- **Data-Adaptive Structure:**

- Root Model predicts which region (sub-model) to access, based on the data's distribution.
- Sub-models provide fine-grained predictions, reducing the need for multiple tree levels

- **Reduced Memory and Faster Lookups:**

- Learned indexes minimize the need for pointers and $\log(n)$ comparisons, using a more compact model.

B+tree vs. Learned Index

	B+ Tree	Learned Index
Structure	Fixed, hierarchical	Data-adaptive, predictive
Search Cost	$\text{Log}(n)$ comparisons	Direct prediction with neighborhood search
Memory Use	High, with pointers per node	Lower, fewer nodes needed
Adaptability	Fixed partitions, no data patterns	Fits patterns, reduces overhead

Learned Index

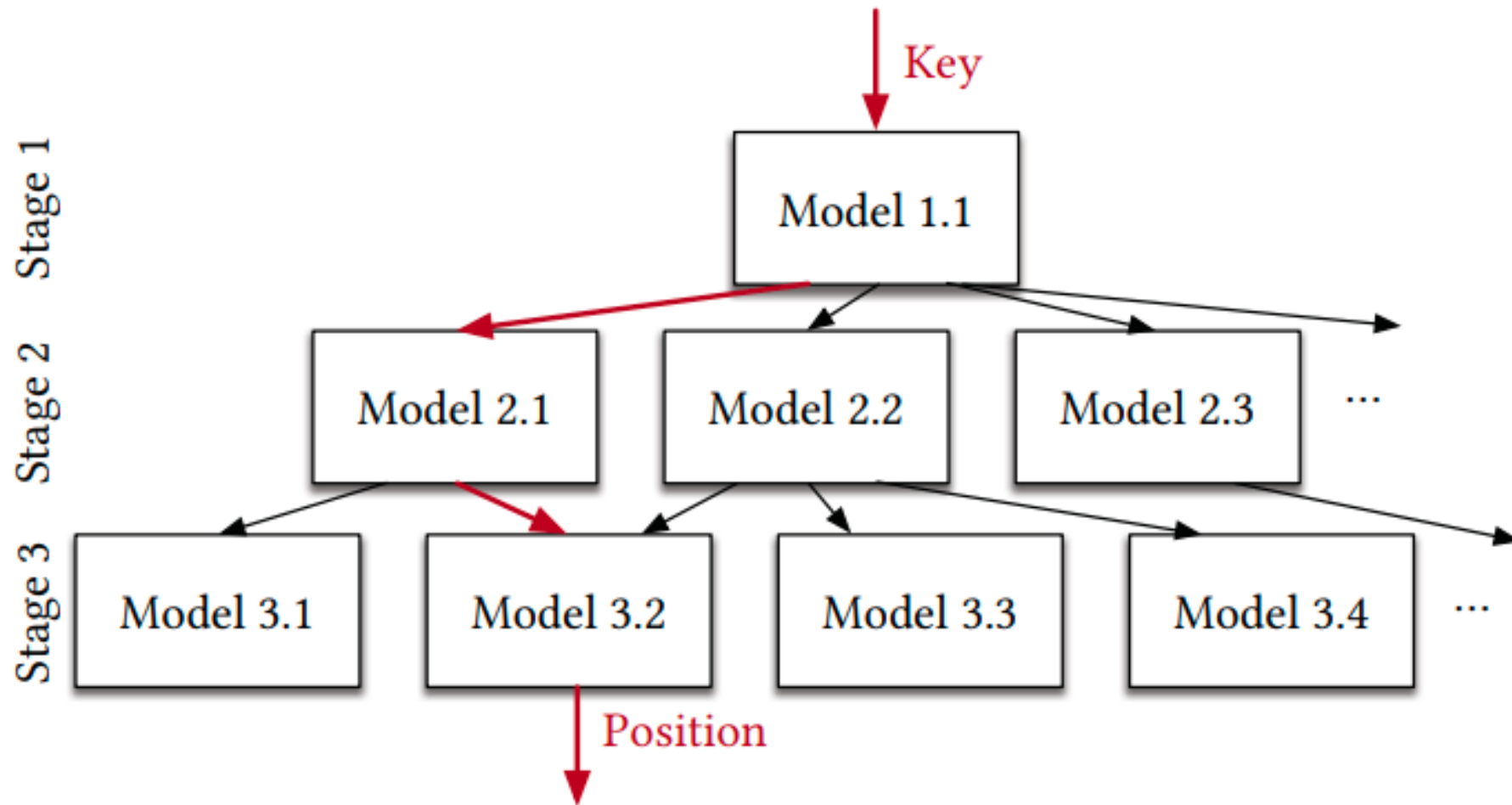
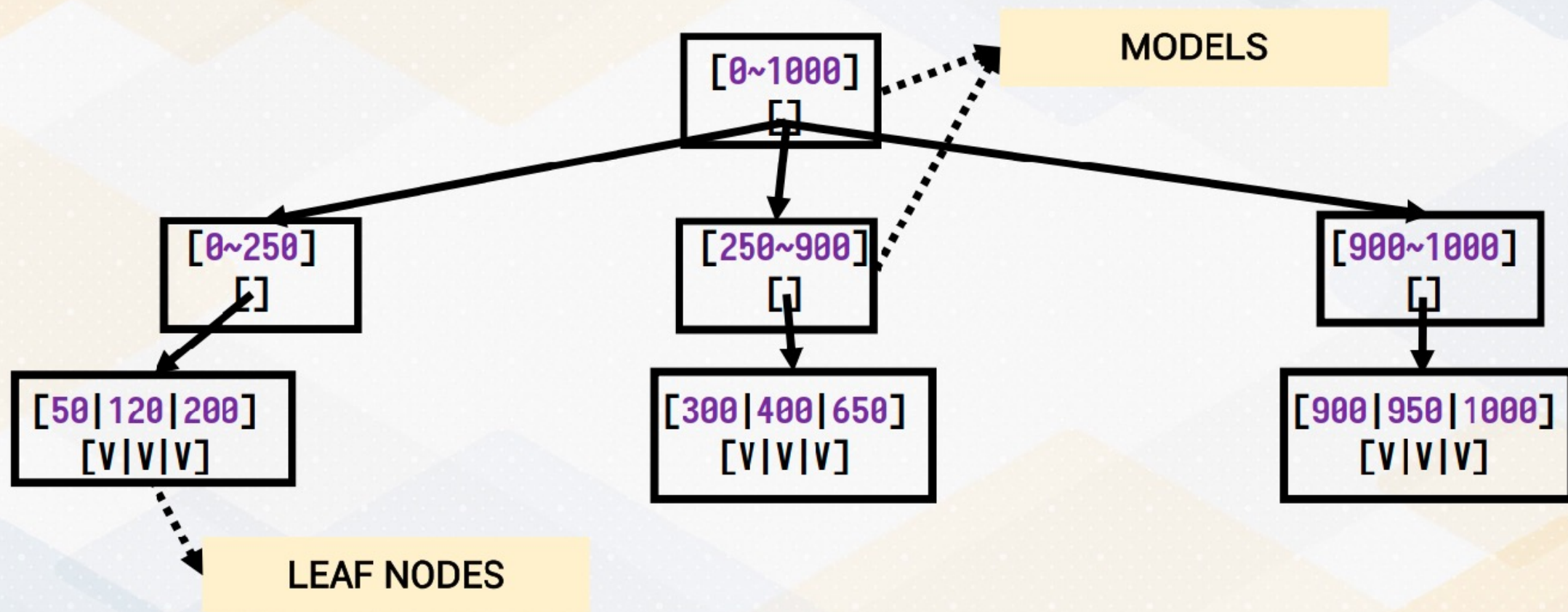


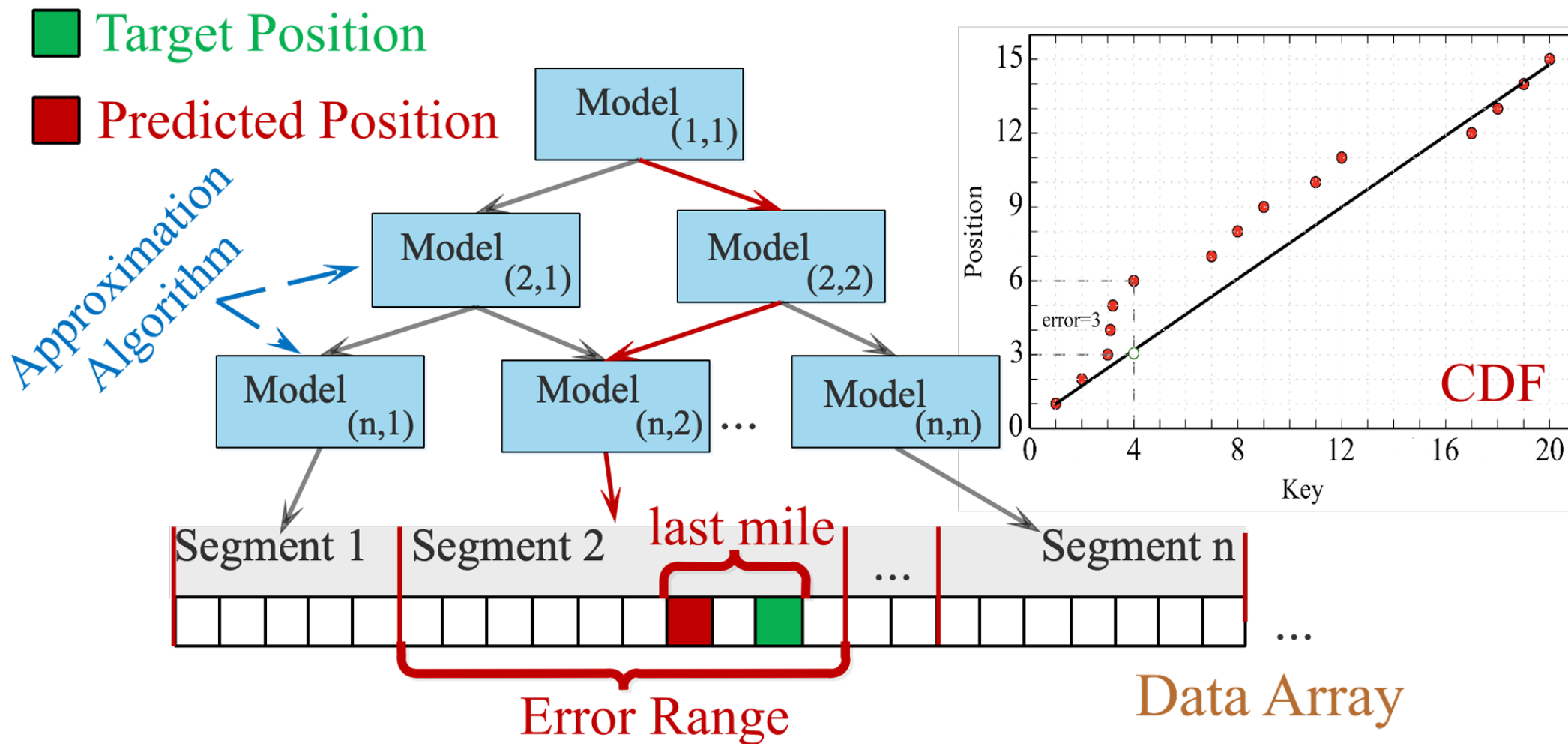
Figure 3: Staged models

Learned Index

Learned Index

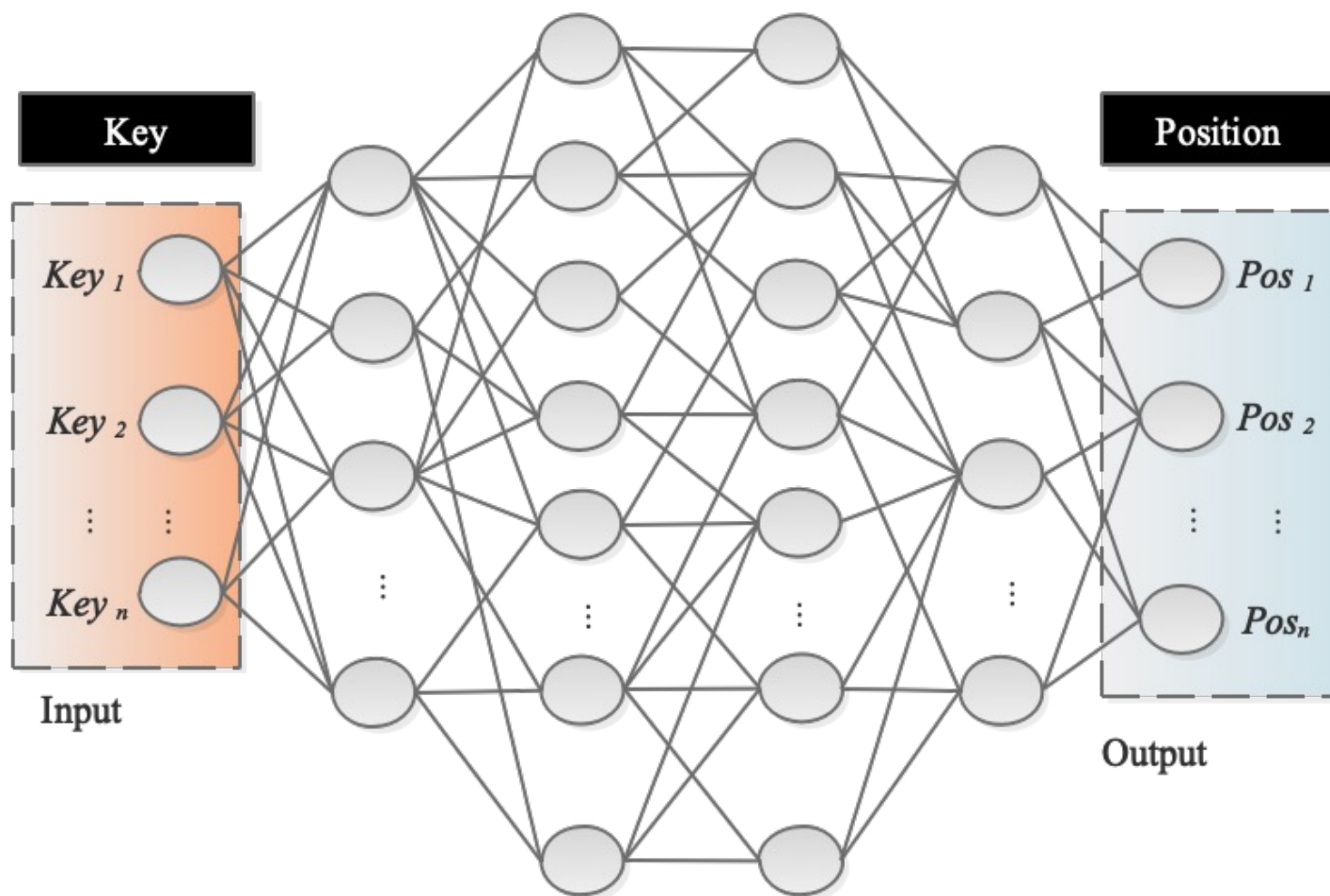


Learned Index

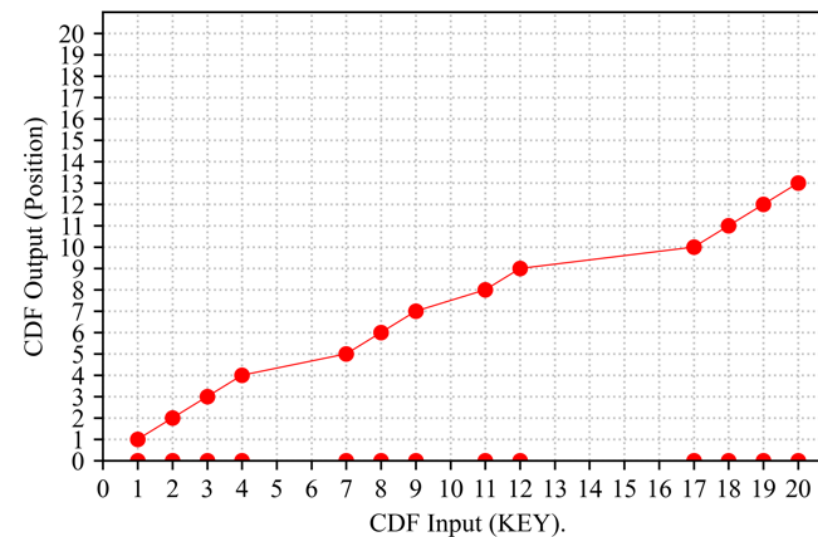


效果： 与B+tree相比， RMI性能无损失， 空间缩小了2-3个数量级。

Learned Index



$$\sum_{(x,y) \in D} (F(x) - y)^2$$



CDF: 表示键值（有序）与存储位置之间的映射特征

Structure of a Learned Index

- **Root Model:** Predicts a range of data positions.
- **Sub-models:** Finer-grained models handle local data within that range.

```
struct Model {  
    double slope = 0.0;  
    double intercept = 0.0;  
    void train(const std::vector<int>& keys, const std::vector<size_t>& positions);  
    size_t predict(int key) const;  
    void print() const;  
};
```

Training the Root and Sub-models

- **Root Model:** Learns a rough mapping of keys to sub-models.
- **Sub-models:** Fine-tune predictions for localized clusters of data.

```
void train(const std::vector<int>& keys, const std::vector<size_t>& positions) {  
    // Calculate mean  
    for (size_t i = 0; i < keys.size(); ++i) {  
        mean_x += keys[i];  
        mean_y += positions[i];  
    }  
    // Compute slope and intercept  
    slope = num / denom;  
    intercept = mean_y - slope * mean_x;  
}
```


Regression

- A statistical method for modeling the relationship between a dependent variable (x) and an independent variable (y).

$$y(x) = \text{slope} * x + \text{intercept}$$

$$\text{Position}(\text{key}) = \text{slope} * \text{key} + \text{intercept}$$

Training Regression Model

- Calculate the mean of keys (x) and positions (y).
- Find the slope and intercept to define the linear relationship.

position	0	1	2	3	4
key	50	100	120	200	250

Training Regression Model

- Trains models by calculating mean, slope, and intercept to fit the data distribution.

```
double num = 0.0, denom = 0.0;
for (size_t i = 0; i < keys.size(); ++i) {
    double x_diff = keys[i] - mean_x;
    num += x_diff * (positions[i] - mean_y);
    denom += x_diff * x_diff;
}
}
slope = num / denom;
intercept = mean_y - slope * mean_x;
```


Training Regression Model

- Numerator accumulate values to compute the covariance of keys and positions.
- Denominator accumulates the variance of keys

$$\text{slope} = \text{num} / \text{denom} = 500 / 20000 = 0.025$$

$$\text{intercept} = \text{mean_y} - (\text{slope} \times \text{mean_x}) = 2 - (0.025 \times 150) = -1.75$$

$$y = 0.025 \times x - 1.75$$

Learned Index

Trained Regression Model

$$y = 0.025 \times x - 1.75$$

position	0	1	2	3	4
key	50	100	120	200	250
y	-0.5	0.75	2	3.25	4.5

Loading Training Data

- Root and sub-models are trained sequentially, with each sub-model learning its local cluster of data.

```
for (int i = 0; i < num_sub_models; ++i) {  
    cluster_keys.clear();  
    cluster_positions.clear();  
    for (size_t j = start; j < end; ++j) {  
        cluster_keys.push_back(sorted_data[j].first);  
        cluster_positions.push_back(j);  
        leaves[i].push_back(sorted_data[j]);  
    }  
    sub_models[i].train(cluster_keys, cluster_positions);  
}
```

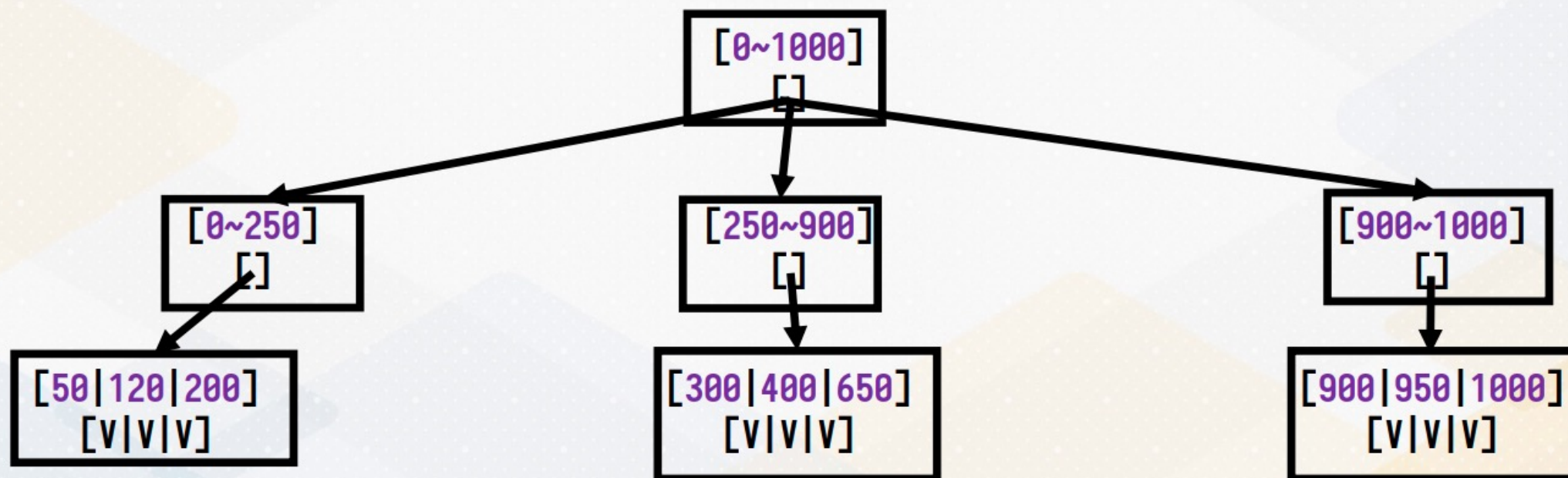
Position Prediction Process

- Root model predicts the sub-model to use.
- Sub-model predicts the approximate position of the key within the data cluster.

```
int sub_model_idx = root_model.predict(key);  
size_t pos = sub_models[sub_model_idx].predict(key);
```


Position Prediction Process

- Root model predicts the sub-model to use.
- Sub-model predicts the approximate position of the key within the data cluster.



Using Trained Regression Model

- Map key to position

```
size_t predict(int key) const {  
    return static_cast<size_t>(std::max(0.0, slope * key + intercept));  
}
```


Neighborhood Search Around Predicted Position

- Searches nearby positions for accuracy, as regression may only predict an approximate location.

```
int search_radius = 5;
int start = std::max(0, int(pos) - search_radius);
int end = std::min(int(leaves[sub_model_idx].size() - 1), int(pos) + search_radius);

for (int i = start; i < end; ++i) {
    if (key == leaves[sub_model_idx][i].first) {
        return leaves[sub_model_idx][i].second;
    }
}
```

Visual Example of Neighborhood Search

- Predicted position = 10
- Search radius = 2

8	9	10	11	12
3	7	15	25	32

Practical Considerations for Neighborhood Search

- **Choosing Radius:**
 - Larger radius increases the likelihood of finding a key but will reduce speed.
 - Radius depends on data distribution and prediction accuracy of the regression model.
- **Optimizing for Real-World Data:**
 - Skewed or clustered data may benefit from dynamic radius adjustment.

Evaluation Metrics

- **Lookup speed:** Learned indexes can transform lookup operations from the typical B-Tree complexity of $O(\log n)$ into near-constant time operations.
- **Memory usage:** Learned indexes use less memory by eliminating redundant pointers and nodes.
- **Prediction accuracy:** By training models to understand the data's cumulative distribution function (CDF), learned indexes achieve high accuracy for point and range queries, even with relatively simple models like linear regression.

Limitation of Learned Index using Regression

- The relationship between keys and positions can be highly non-linear, with gaps increasing at an exponential rate.
- A linear regression model would struggle to capture these intervals accurately.

0	1	2	3	4	5	6	7	8
10	20	45	100	200	350	600	1000	1500

Non-Linearity

- **Data Distribution:** Real-world datasets often exhibit complex, non-linear relationships that cannot be accurately modeled by a simple line or function.
- **Examples:** Customer purchase behavior, seasonal data patterns, and geographic data often show non-linear trends.
- **Limitations of Linear Models:** Linear regression can only capture straightforward relationships.
- For data that fluctuates unpredictably or has clusters and non-uniform distributions, linear models fall short.

Neural Network

- A neural network can capture non-linear data patterns by learning the complex mapping.
- With multiple hidden layers and neurons, they can model intricate relationships that simpler models miss.

Abstract Model Class

- Defines a general model interface with train, predict, and print methods.

```
struct Model {  
    virtual ~Model() {}  
    virtual void train(const std::vector<int>& inputs, const std::vector<double>& targets) = 0;  
    virtual double predict(int x) const = 0;  
    virtual void print() const = 0;  
};
```

SimpleNeuralNetwork Class

- A single-layer neural network model with input_weights, output_weights, input_bias, and output_bias.
- hidden_neurons: Defines the number of neurons in the hidden layer.

```
class SimpleNeuralNetwork : public Model {  
    std::vector<double> input_weights;  
    std::vector<double> output_weights;  
    double input_bias, output_bias;  
    int hidden_neurons;  
    // other variables and helper functions...  
};
```


Network Structure

- **Input Layer:** 1 neuron for the key.
- **Hidden Layer:** 10 neurons, with weights and biases initialized randomly.
- **Output Layer:** 1 neuron for the predicted position.

```
SimpleNeuralNetwork nn(10); // 10 hidden neurons
```

Training the Neural Network for Learned Index

- **Data Normalization:**
 - Inputs are normalized to $[0, 1]$ to enhance training performance.

```
void train(const std::vector<int>& inputs, const std::vector<double>& targets) {  
    for (int epoch = 0; epoch < epochs; ++epoch) {  
        for (size_t i = 0; i < inputs.size(); ++i) {  
            // Forward and backward pass...  
            output_bias += learning_rate * delta_output;  
            input_bias += learning_rate * delta_hidden;  
        }  
    }  
}
```


Training the Neural Network for Learned Index

- **Forward Pass:** Calculates hidden layer outputs using sigmoid activation, then sums outputs for final prediction.
- **Backward Pass:** Adjusts weights and biases based on error between prediction and actual target.
- **Epochs and Learning Rate:** Trains over multiple epochs with a small learning rate to improve prediction accuracy.

Training the Neural Network for Learned Index

- **Input Normalization:**
- Normalize each key by dividing by the maximum key (1500) to improve training.
- **Input to Hidden Layer:**
- Each hidden neuron computes its output using:

$$h_j = \sigma(x \cdot w_{i,j} + b_j)$$

- Here, x is the normalized key, $w_{i,j}$ is the weight for hidden neuron j , and b_j is the bias for neuron j .

Training the Neural Network for Learned Index

- Hidden to Output Layer:
- Compute the final predicted position by summing up:

$$y = \sum (h_j \cdot w_{h,j}) + b_o$$

- Here, h_j : Output of hidden neuron j ; w_{hj} : weight from hidden neuron j to output; b_o : output bias.

$$\text{Loss} = \frac{1}{n} \sum (\text{predicted position} - \text{actual position})^2$$

Recursive Model Index

- Root and Sub-Models in the RMI Using Neural Networks

```
void initModels(bool use_neural_network) {  
    if (use_neural_network) {  
        root_model = std::make_unique<SimpleNeuralNetwork>();  
        for (int i = 0; i < 4; ++i) {  
            sub_models.push_back(std::make_unique<SimpleNeuralNetwork>());  
        }  
    }  
}
```


Using Neural Network Prediction for Key Search

- The root_model predicts which sub-model to use.
- The selected sub-model then predicts the position of the key within its data cluster.

```
int sub_model_idx = root_model->predict(key);  
size_t pos = sub_models[sub_model_idx]->predict(key);
```

Learned Index

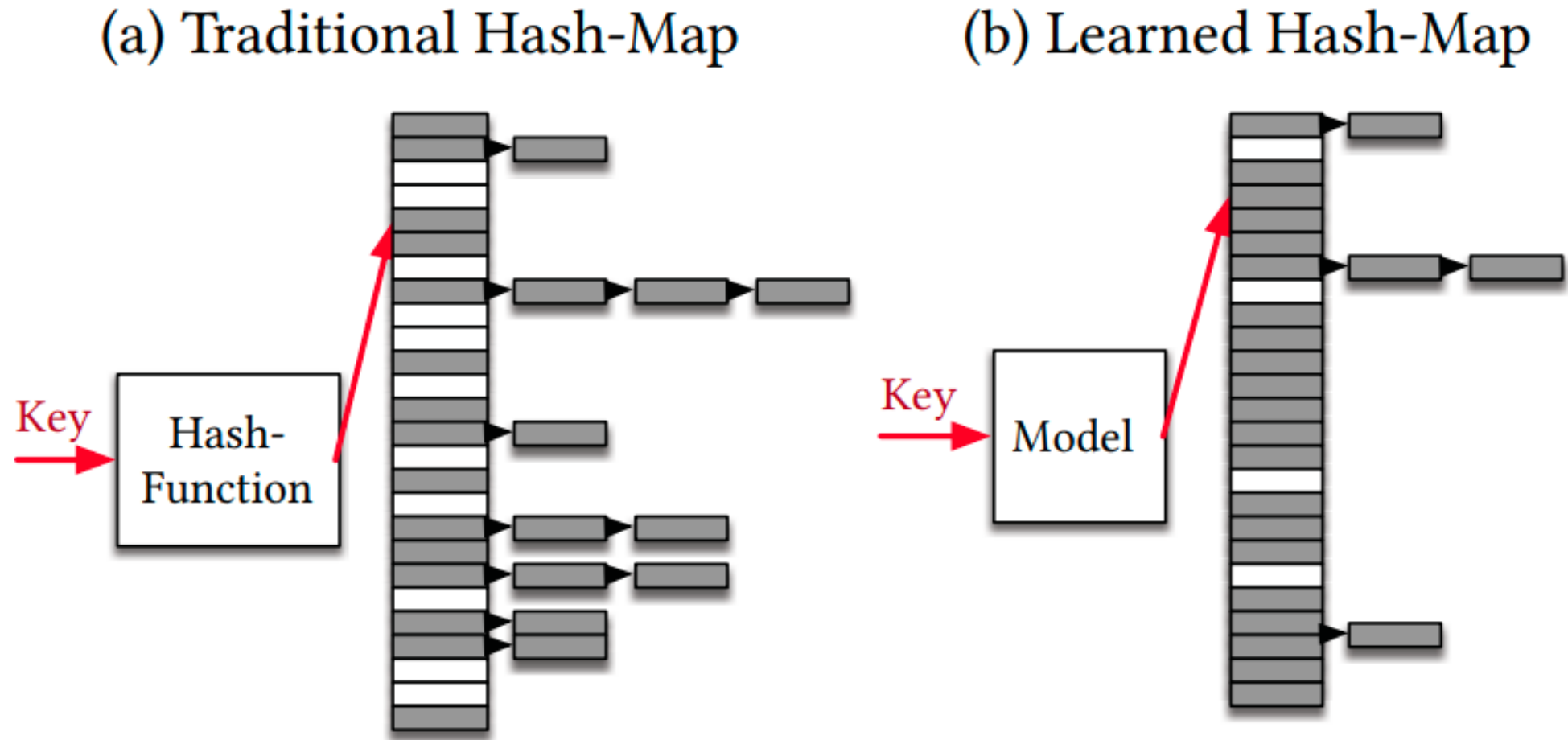


Figure 7: Traditional Hash-map vs Learned Hash-map

Learned Index

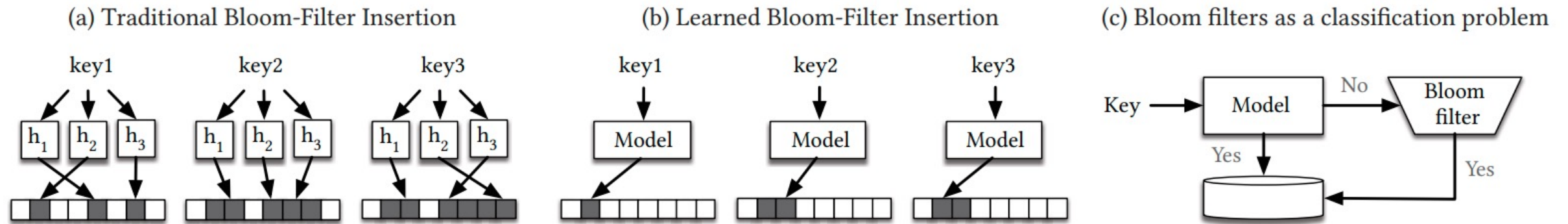


Figure 9: Bloom filters Architectures

Kraska T, Beutel A, Chi E H, et al. The case for learned index structures[C]//Proceedings of the 2018 international conference on management of data. 2018: 489-504.

Learned Index

Approximation function

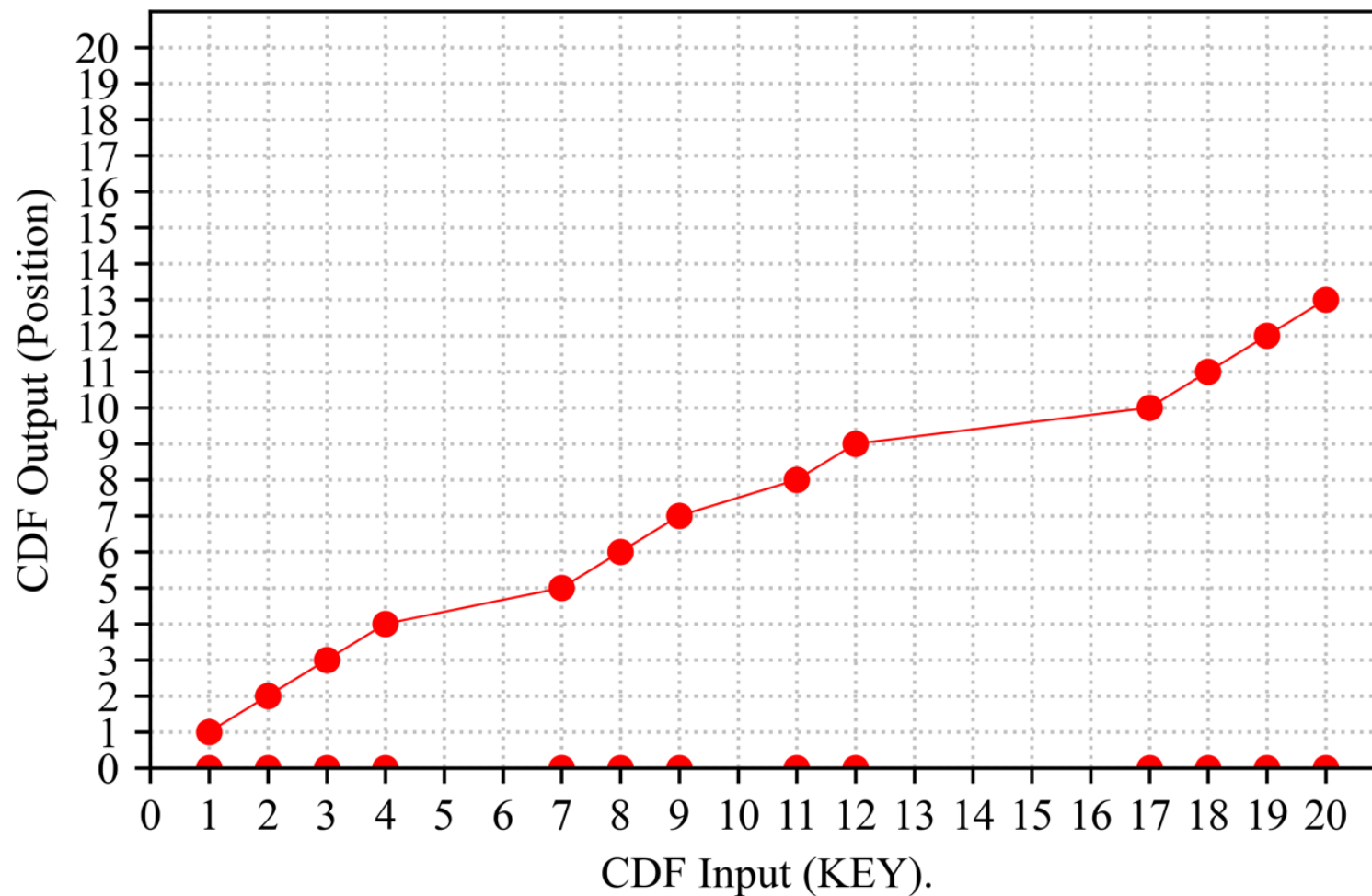
Name	Form	Parameters
Linear	$f(x) = ax + b$	a, b
Log-linear	$f(x) = \exp(ax + b)$	a, b
Cubic	$f(x) = ax^3 + bx^2 + cx + d$	a, b, c, d
Radix	$f(x) = x \gg a$	a
Normal	$f(x) = \frac{1}{2} \times \left(1 + \operatorname{erf} \left(\frac{x-a}{b \times \sqrt{2}} \right) \right)$	a, b

Table 1: A subset of available model types

不同CDF情况下，如何找到最优拟合算法仍是悬而未决的问题。

Learned Index

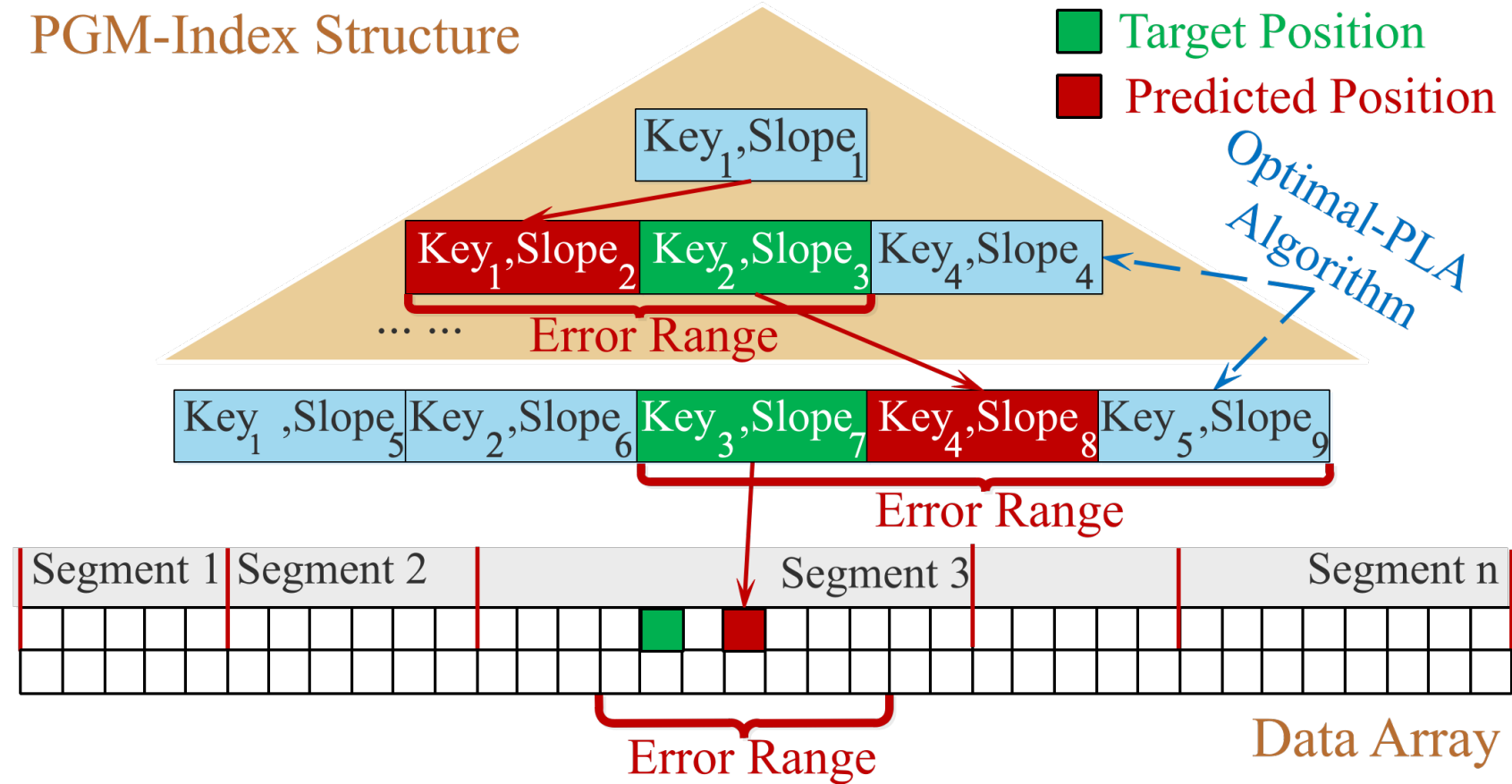
CDF拟合算法之: 分段线性函数



Learned Index

PGM-Index (LRS + Optimal-PLA)

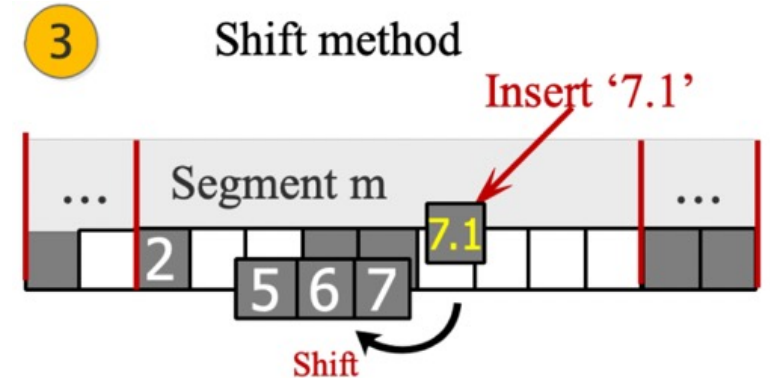
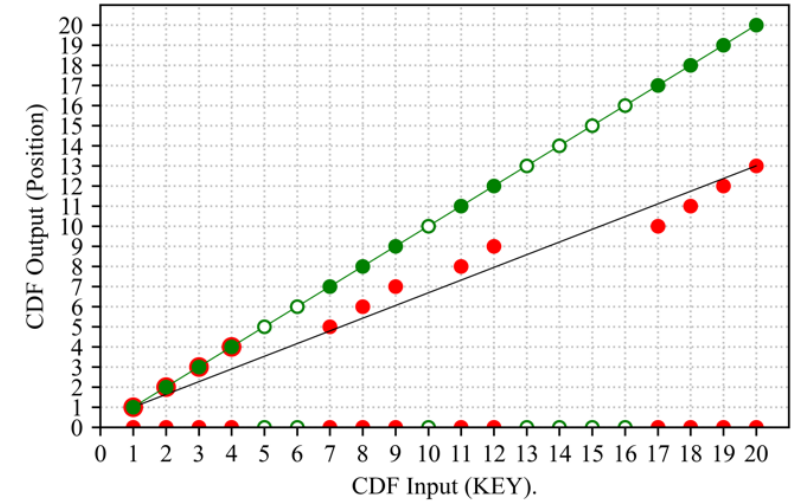
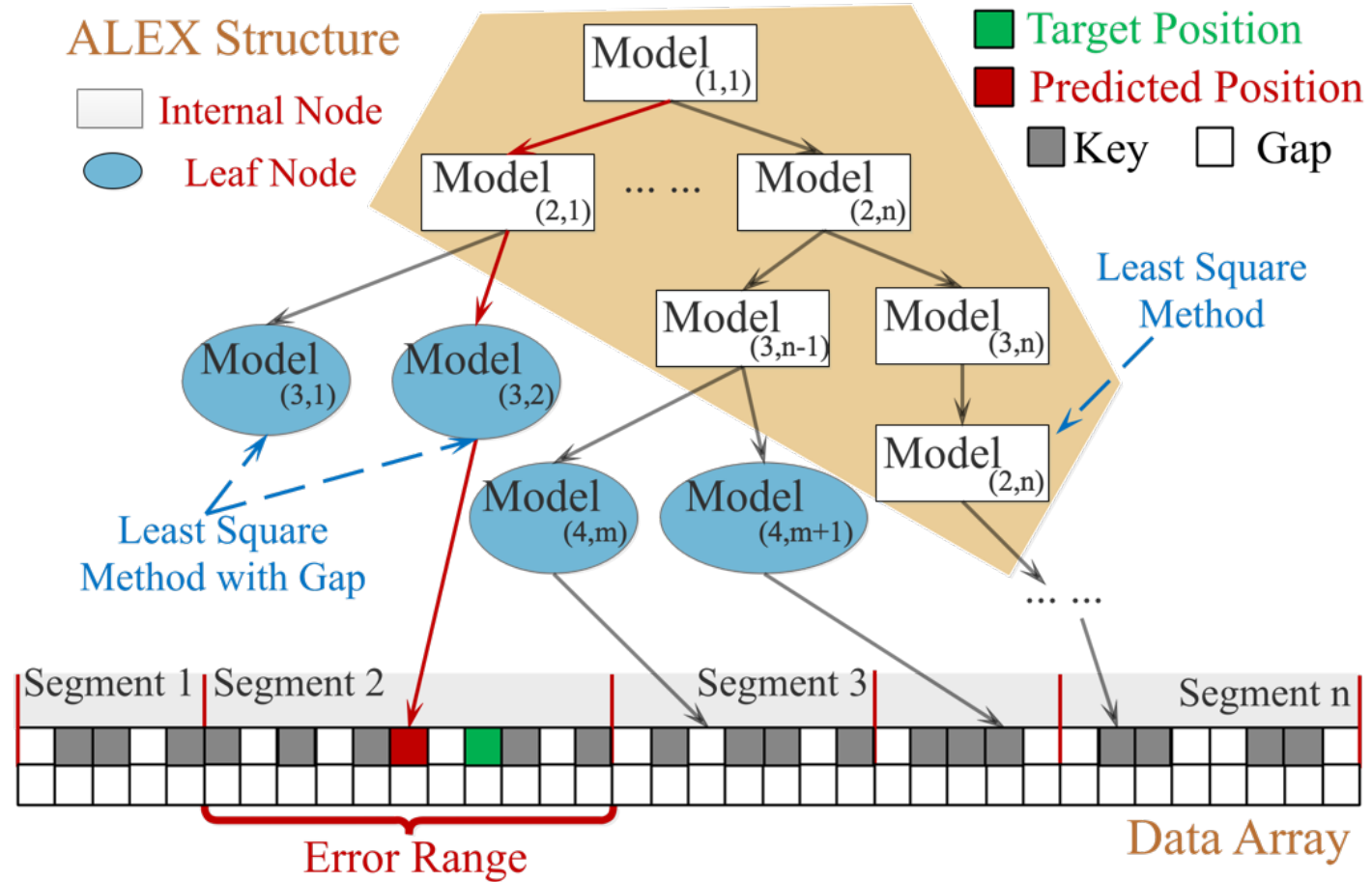
PGM-Index Structure



Paolo Ferragina and Giorgio Vinciguerra. 2020. **The PGM-index: a fully-dynamic compressed learned index with provable worst-case bounds**. Proceedings of the VLDB Endowment 13, 8 (2020), 1162–1175.

Learned Index

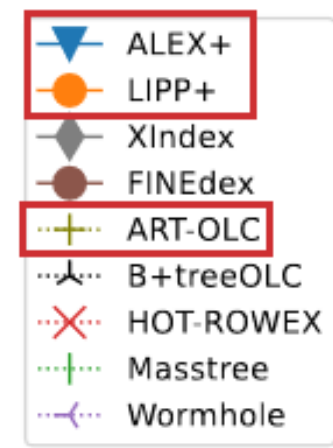
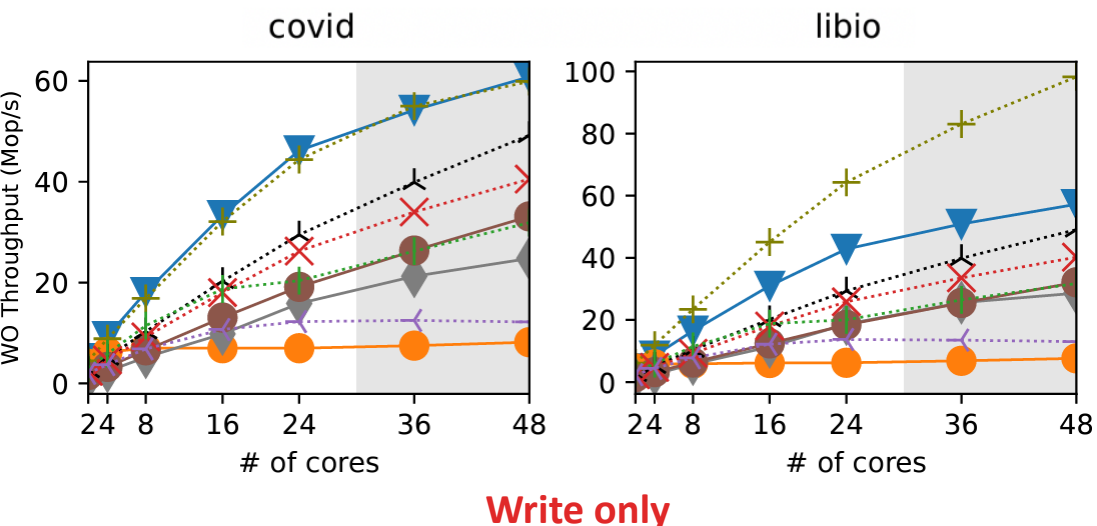
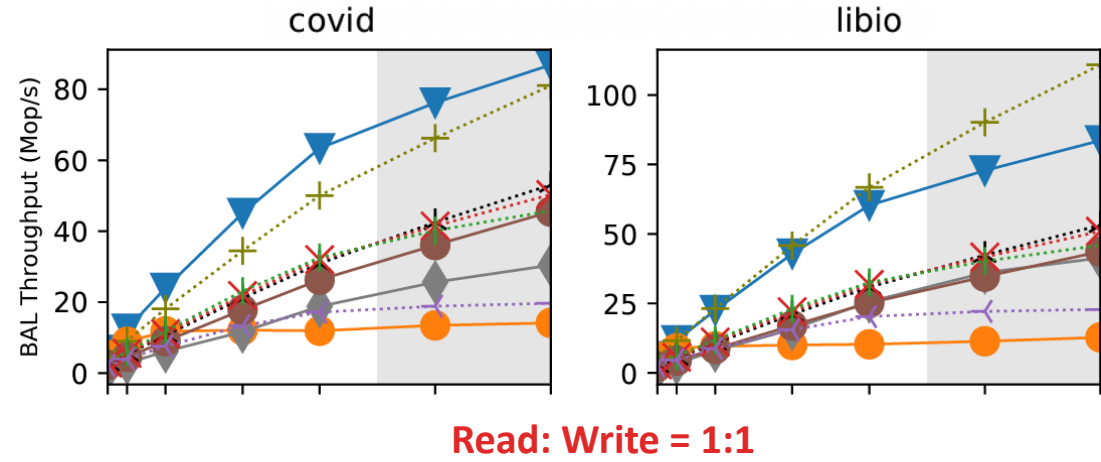
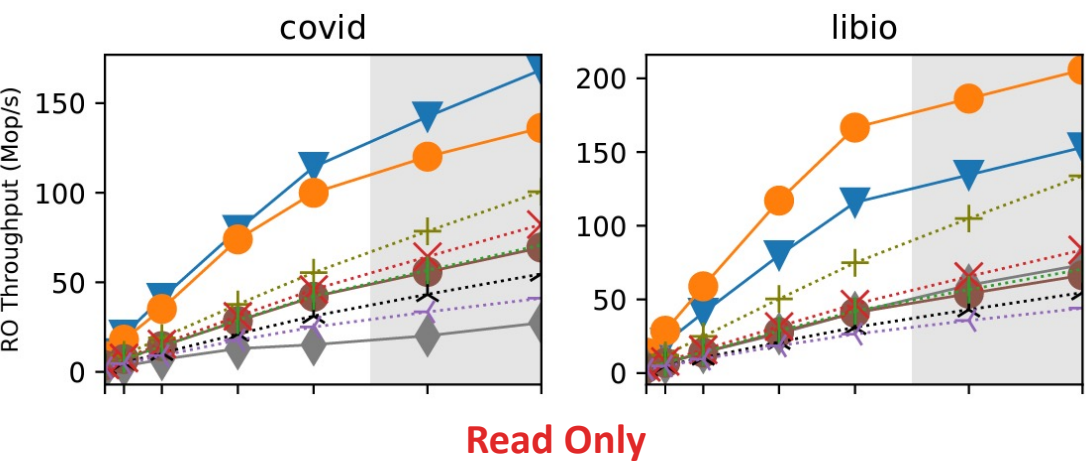
ALEX (model-based+Shift, Mod+S)



Learned Index

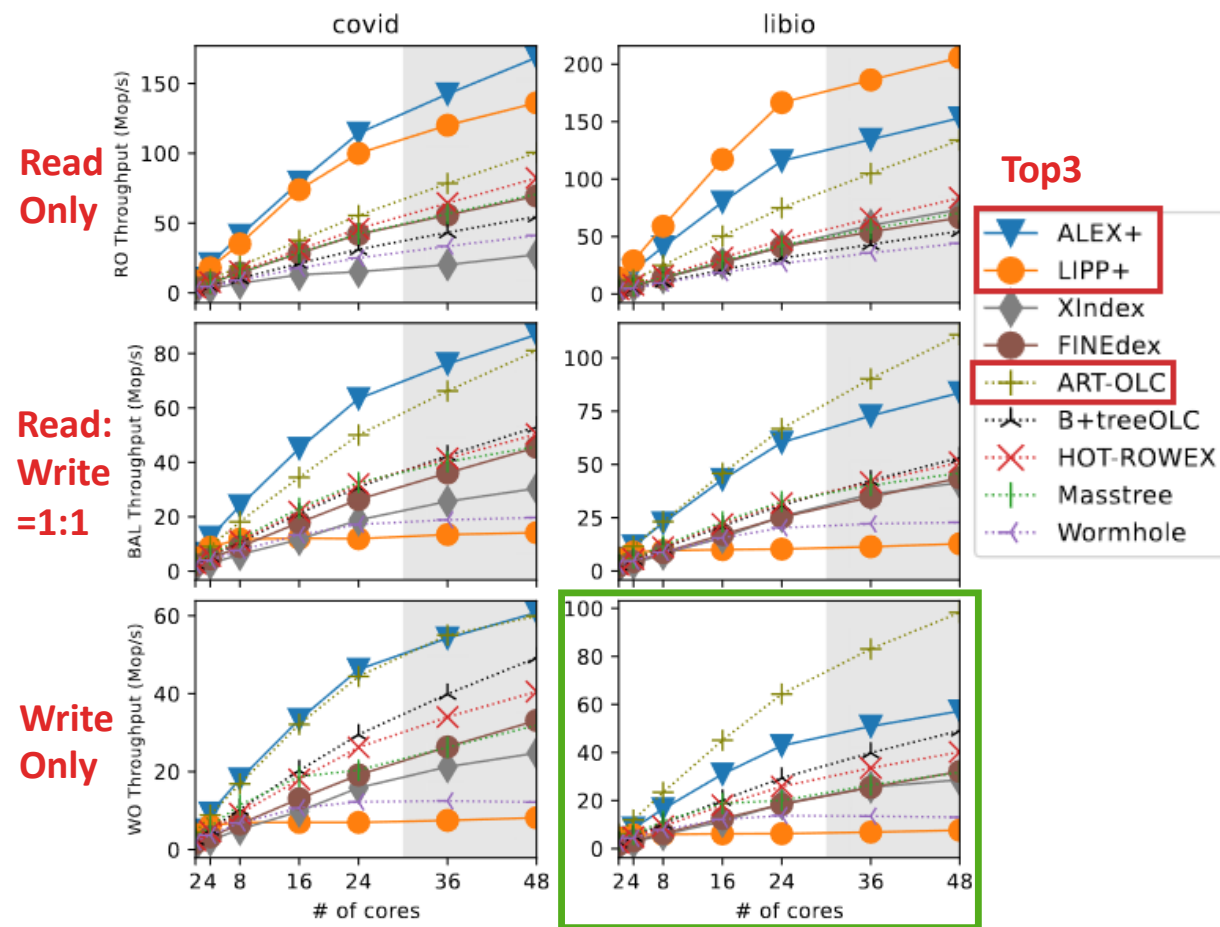
方案	构建代价	更新支持	误差界	典型用途
RMI	离线训练	否	无保证	研究原型、只读
ALEX	在线	是（节点分裂）	自适应	内存 KV、写读混合
PGM	$O(N)$	是（LSM 合并）	有 worst-case	通用学习型索引

学习型索引在大多数工作负载中均优于传统索引

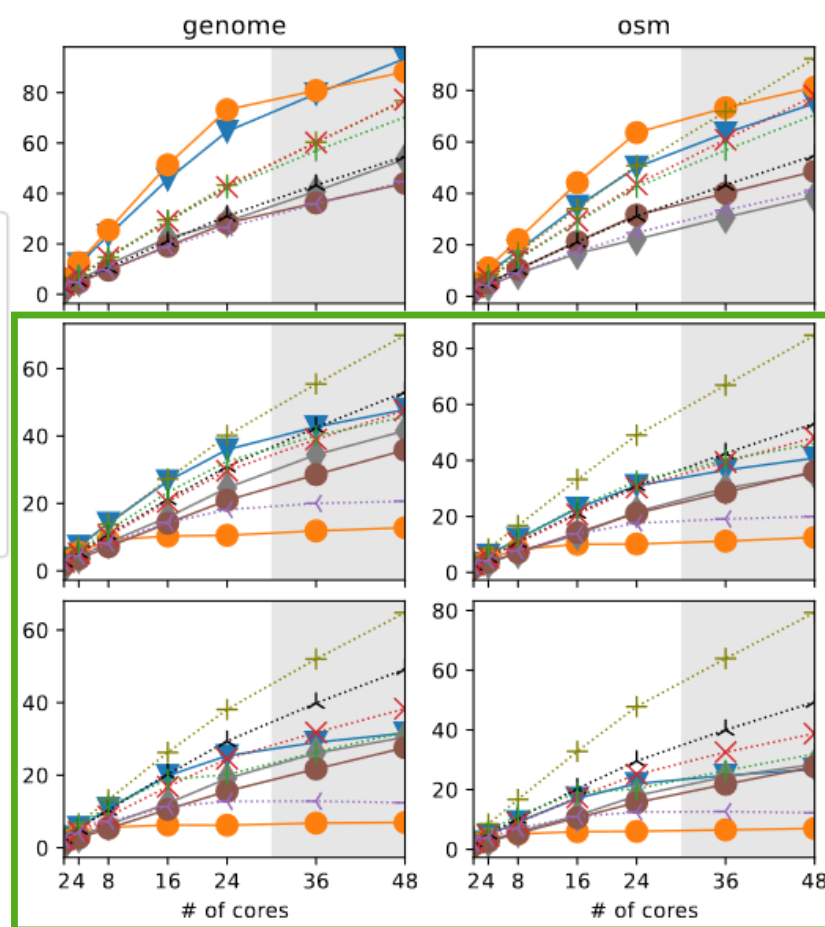


学习型索引在hard情况下write only性能较差

Easy Datasets – Easy for Linear Model to Fit



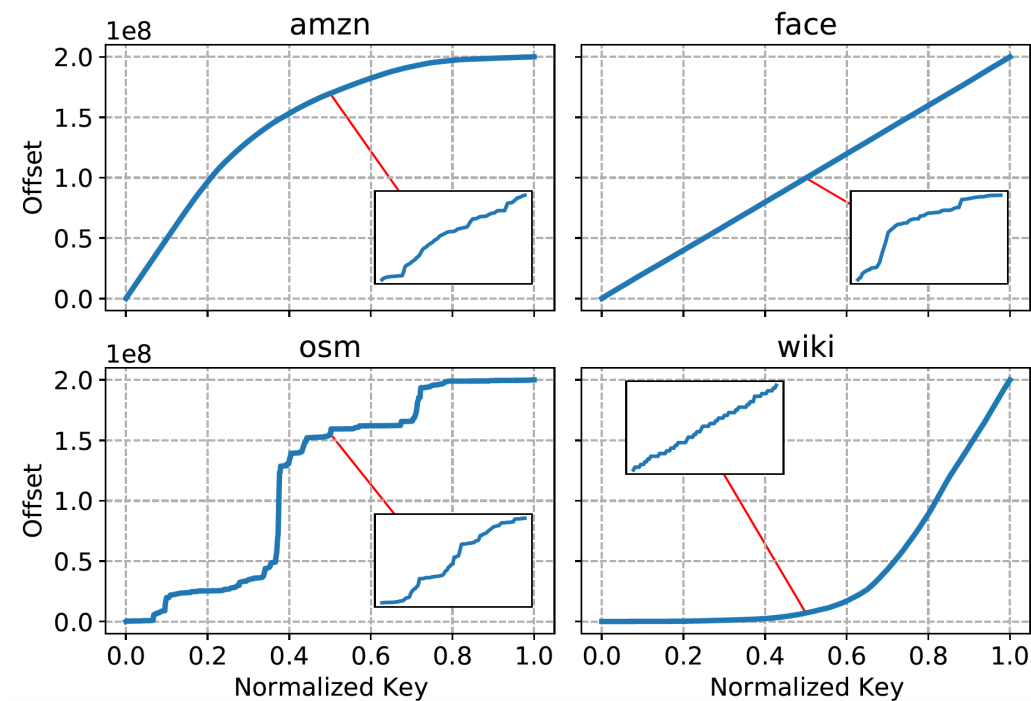
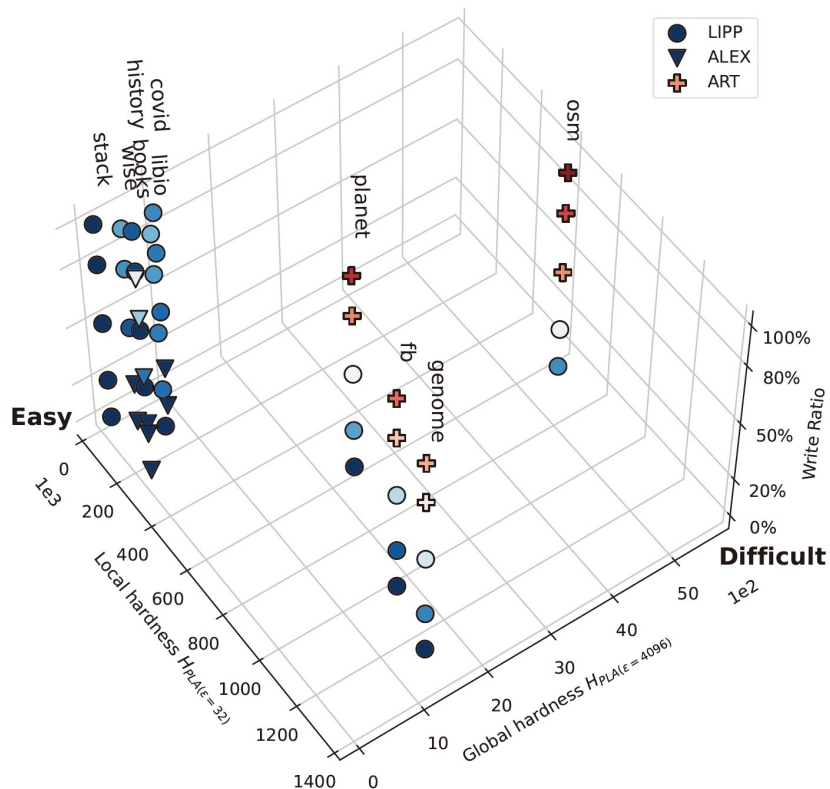
Hard Datasets – Hard for Linear Model to Fit



Write only 和 CDF hard模式下学习索引性能不如传统索引

学习型索引在hard情况下write only性能较差

- 简单数据集 + 读为主的工作负载下，学习索引领先传统索引
 - 现实中大部分数据集都是简单数据集
 - 困难数据集下学习索引难以拟合CDF特征



学习索引的并发性能较差，可拓展性不足

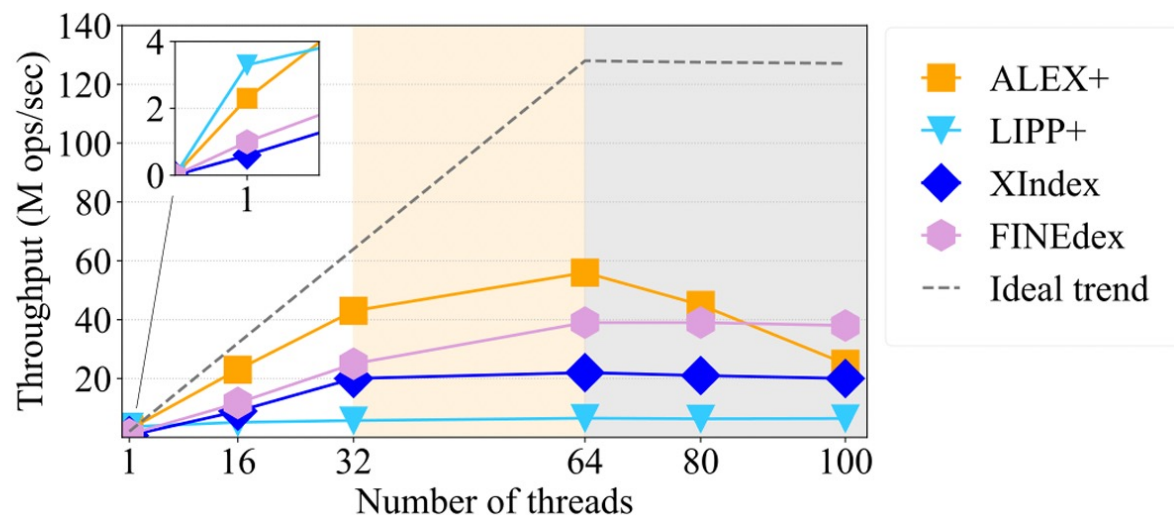
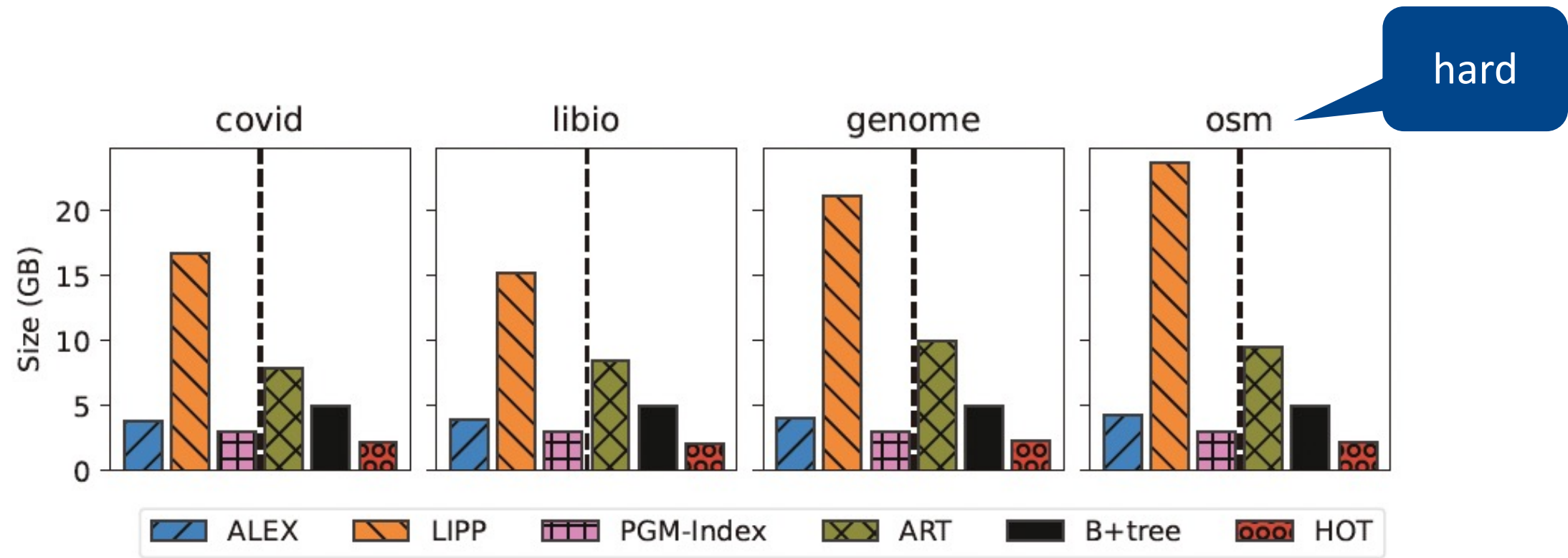


Figure 1: Write-only performance of state-of-the-art learned indexes on the FACE dataset [11]. The evaluation is conducted on a two-socket machine with two 16-core CPUs.

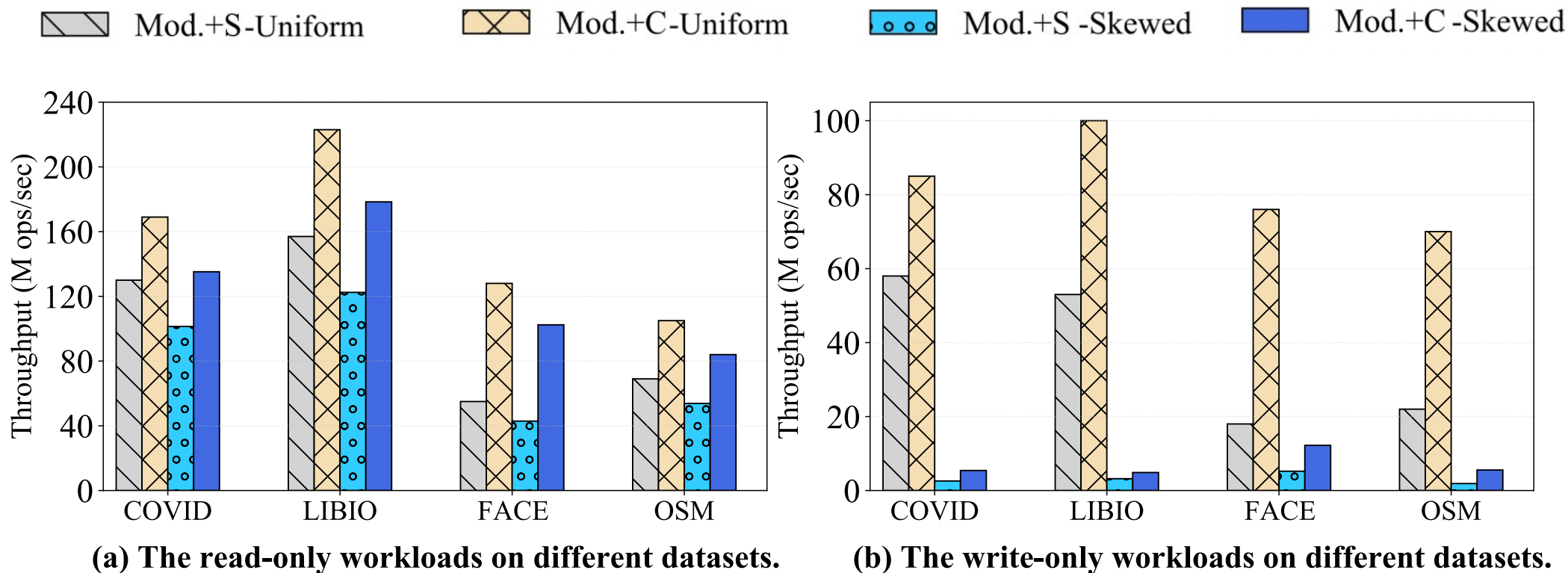
单线程表现最优的学习索引，多线程表现劣化

部分可更新学习索引的空间开销较大



LIPP等可更新学习索引的空间开销较大，hard数据集下空间开销更严重。

倾斜工作负载场景下学习索引性能较差



观察 5: 相比于非倾斜工作负载, 倾斜的只读工作负载会导致 20% 到 30% 的性能损失。

观察 6: 相比于非倾斜工作负载, 倾斜的只写工作负载会导致高达 70% 至 85% 的性能损失。

学习索引面临的挑战

- 1、学习型索引在hard情况下write only性能较差;
- 2、学习索引的并发性能较差, 可拓展性不足;
- 3、可更新学习索引的空间开销较大;
- 4、倾斜工作负载场景下学习索引性能较差。

Thanks.